郑 重 声 明

本人呈交的学位论文，是在导师的指导下，独立进行研究工作所取得的成果，所有数据、图片资料真实可靠。尽我所知，除文中已经注明引用的内容外，本学位论文的研究成果不包含他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确的方式标明。本学位论文的知识产权归属于培养单位。

本人签名：_____　　日期：

摘　要

随着电子商务的快速发展，传统单体架构逐渐暴露出在面对用户需求和复杂业务场景时，存在维护困难和扩展性差的问题，尤其在高用户量和交易量下，传统架构无法满足现代电商平台对高可用性和高并发处理的需求。为了解决这些问题，本论文设计并实现了一个基于微服务架构的购物商城系统，涵盖购物、用户管理、订单处理、商品展示等功能，支持电商平台在业务扩展和技术升级中的灵活性和可扩展性。

系统采用前后端分离的模式，前端使用 Vue.js 框架，保证了丰富的用户交互和高效开发。后端基于 Java、Spring Cloud Alibaba 和 SpringBoot 框架，充分发挥微服务架构的优势，实现了系统模块的独立部署和扩展。数据库使用 MySQL，优化了数据存取效率；缓存采用 Redis，提升了响应速度，减轻了数据库压力。为实现异步处理和解耦，系统引入 RabbitMQ 作为消息中间件，确保了高效的服务间通信和任务调度。

通过微服务架构，本设计不仅解决了传统架构的性能瓶颈，还提升了系统的灵活性和可扩展性，能够在高并发、大流量的情况下稳定运行，满足电商平台的日益增长需求。同时，系统的模块化设计简化了后续功能拓展和维护。

关键字：购物商城；微服务；微信小程序；SpringBoot 框架

Abstract

With the rapid development of e-commerce, traditional monolithic architectures have gradually revealed problems such as difficult maintenance and poor scalability when dealing with user demands and complex business scenarios, especially under high user volumes and transaction volumes. Traditional architectures cannot meet the requirements of modern e-commerce platforms for high availability and high concurrent processing. To address these issues, this paper designs and implements a shopping mall system based on a microservice architecture, covering functions such as shopping, user management, order processing, and product display, supporting the flexibility and scalability of e-commerce platforms in business expansion and technological upgrades.

The system adopts a front-end and back-end separation model. The front-end uses the Vue.js framework to ensure rich user interaction and efficient development. The back-end is based on Java, Spring Cloud Alibaba, and SpringBoot frameworks, fully leveraging the advantages of the microservice architecture to achieve independent deployment and expansion of system modules. MySQL is used for the database to optimize data access efficiency, and Redis is adopted for caching to enhance response speed and reduce database pressure. To achieve asynchronous processing and decoupling, RabbitMQ is introduced as a message middleware to ensure efficient communication and task scheduling between services.

Through the microservice architecture, this design not only resolves the performance bottlenecks of traditional architectures but also enhances the flexibility and scalability of the system, enabling stable operation under high concurrency and large traffic, meeting the growing demands of e-commerce platforms. Meanwhile, the modular design of the system simplifies subsequent function expansion and maintenance.

Keywords: Shopping mall ; Microservices ; WeChat program ; SpringBoot

framework

目录

摘　要.....I
Abstract.....II
1 前言.....1
1.1 研究背景.....1
1.2 研究目的和意义.....2
2 系统相关技术概述.....3
2.1 SpringBoot 框架.....3
2.2 SpringCloud 框架.....5
2.3 Mysql 数据库.....6
2.4 Redis 缓存.....7
2.5 RabbitMQ 消息队列.....8
2.6 SpringSecurity 权限校验框架.....8
2.7 Minio 存储技术.....10
3 商城购物系统需求分析.....11
3.1 后台服务端功能需求分析.....11
3.1.1 订单功能需求分析.....11
3.1.2 用户功能需求分析.....12
3.1.3 系统功能需求分析.....12
3.1.4 产品功能需求分析.....13
3.1.5 门店功能需求分析.....14
3.2 微信端功能需求分析.....15
3.2.1 前台门户功能需求分析.....15
3.2.2 用户中心功能需求分析.....16
3.2.3 购物车需求分析.....17
3.2.4 订单功能需求分析.....18
4 数据库设计.....19
4.1 Linux 里安装数据库.....19

4.2 数据库逻辑模型设计.....20
4.2.1 系统业务表.....20
4.2.2 商品业务表.....21
4.2.3 门店业务表.....22
4.2.4 会员业务表.....22
4.2.5 购物车业务表.....23
4.2.6 订单业务表.....24
5 商城购物系统设计.....25
5.1 系统架构设计.....25
5.2 系统功能模块设计.....26
5.2.1 网关微服务设计.....26
5.2.2 认证授权微服务设计.....28
5.2.3 订单微服务设计.....29
5.2.4 用户微服务设计.....31
5.2.5 购物车微服务设计.....32
5.2.6 系统微服务设计.....32
5.2.7 搜索微服务设计.....34
5.2.8 产品微服务设计.....34
5.2.9 门店微服务设计.....36
6 商城购物系统实现.....37
6.1 开发环境.....37
6.2 通用功能实现.....37
6.3 微信端核心业务实现.....41
6.3.1 商品首页功能实现.....41
6.3.2 分类功能实现.....46
6.3.3 商品详情功能.....49
6.3.4 购物车功能实现.....55
6.3.5 个人中心功能实现.....58
6.4 后台管理端核心业务实现.....63
6.4.1 平台登录页面.....63

6.4.2 用户管理模块实现.....64
6.4.3 订单管理模块实现.....65
6.4.4 系统管理模块设计.....67
6.4.5 产品管理模块设计.....76
6.4.6 门店管理模块设计.....81
7.总结与展望.....86
参考文献.....87
致谢.....89

1 前言

1.1 研究背景

随着经济的迅速发展，我们已经进入了互联网时代。互联网的普及彻底改变了购物的方式，开启了全新的购物模式。电子商务作为这一时代的重要产物，拥有广阔的前景，并为消费者带来了前所未有的购物体验。

电子商务，顾名思义，是通过电子手段进行的商务活动。它将传统的实体市场进行了全面改革，提供了从信息交流到商品交易的全方位服务，具有电子化和数字化的特点。电子商务不仅仅是商贸活动的创新，它把市场、商务和交易流程紧密结合，形成了一个全新的商业模式。从电子商务的诞生开始，人们的生活方式也随之发生了深刻变化。

自 20 世纪 90 年代开始，互联网技术的不断发展，使得信息传播速度加快，信息交流成本降低，为电子商务的崛起提供了坚实的基础，电子商务开始进入了快速发展的时期，2000 年后，出现了更多新的电商平台，如淘宝，京东等，与此同时，移动设备和智能手机的普及使得手机购物成为一股新兴的趋势，进一步推动了电子商城的发展。手机不再仅仅是通信工具，而是成为了人们获取信息、进行消费的重要平台。

网民数量的激增也是电子商城发展的关键因素。随着互联网的普及和智能手机价格的不断下降，越来越多的网民开始接触和使用电子商城。据相关数据显示，中国网民数量已超过 9 亿人，其中手机网民占比高达 80%以上。如此庞大的网民基础，为电子商城提供了广阔的市场空间。

电商的好处是毋庸置疑的，在用户的角度看，使得广大用户可以不受地区约束，不论是南方还是北方，甚至国外，都可以随时找到各地的商品去购买，提供了便利的购物体验，这种便利性不仅为消费者节省了时间和精力，还提高了购物的灵活性和自由度。对于商家，只要有资格允许，每个人都可以成为商家，不再受地域的局限。打破了传统的销售局限，帮助企业扩大了市场范围，传统实体店的销售仅限于本地或者周边地区，而电子商务则能跨越地域和国界，触达全球市场。通过电子平台，企业可以将产品或服务推向更广阔的消费者群体，促进销售增长和市场份额的扩大。

在未来，随着技术的不断创新和市场的不断扩展，电子商城将继续保持快速发展的势头，为消费者提供更加便捷、高效、个性化的购物体验[1]。

1.2 研究目的和意义

互联网的快速发展和电子商务的普及，暴露了单体架构的种种问题。在单体架构中，所有模块都被集成在一个应用程序内，系统规模的扩大使得维护变得越来越困难，且无法对某些模块进行独立扩展。此外，一旦某个模块发生故障，可能导致整个系统崩溃，系统的可用性和可靠性都面临极大挑战。

单体架构诸多局限性变得明显。为了应对复杂的业务需求和快速变化的市场环境，越来越多的系统正在向微服务架构转型，以提高系统的可维护性、扩展性和可靠性。购物商城作为电商系统的重要组成部分，承载着大量的用户、订单、支付等核心业务功能，因此对系统的高可用性、可扩展性和维护性提出了更高的要求。在此背景下，微服务架构逐渐成为大型系统的首选技术架构，基于微服务的购物商城系统的研究具有重要的理论和现实意义[2]。

2 系统相关技术概述

2.1 SpringBoot 框架

购物商城的开发采用了 SpringBoot 框架，而 SpringBoot 的设计目的是为了简化 Spring 应用的初始搭建和开发过程。如果想深入了解 SpringBoot 框架，首先需要对 Spring 框架有所了解。

Spring 是一个广泛使用的 Java EE 轻量级开源框架，由“Spring 之父”Rod Johnson 提出并创建。它的主要目标是简化 Java 企业级应用的开发难度和缩短开发周期。Spring 的应用不仅仅局限于服务器端开发，从简化开发、提高可测试性和实现松耦合的角度来看，任何 Java 应用都能从 Spring 框架中获得益处,Spring 框架不仅提供自身的功能，还具备与其他技术和框架整合的能力。

在不同的语境中，Spring 所代表的含义是不同的。下面分别从“广义”和“狭义”两个角度，对 Spring 进行介绍。

广义的 Spring：指的是 Spring 技术栈，泛指以 Spring Framework 为核心的 Spring 技术栈。经过十多年的发展，Spring 已经不再仅仅是一个应用框架，而是演变为一个由多个不同子项目（模块）组成的成熟技术体系，包括 Spring Framework、Spring MVC、SpringBoot、Spring Cloud、Spring Data、Spring Security 等。其中，Spring Framework 是其他子项目的基础。

狭义的 Spring：指的是 Spring Framework，通常将它称为 Spring 框架。Spring 框架是一个分层、面向切面的轻量级解决方案，专为 Java 应用程序设计，它是 Spring 技术栈的核心和基础，旨在解决企业级应用开发中的复杂性[3]。

Spring 有两个最核心模块： IOC 和 AOP。

IOC：Inverse of Control 的简写，译为“控制反转”，指把创建对象过程交给 Spring 进行管理。

AOP：Aspect Oriented Programming（AOP）即“面向切面编程”。AOP 主要用于封装多个类之间的公共行为，将那些与业务无关但被多个业务模块共同使用的逻辑进行抽取，从而减少系统中的重复代码，降低模块之间的耦合度。此外，AOP 还能够解决一些系统层面的问题，如日志、事务、权限等。

Spring 其他模块如图 2.1 所示：

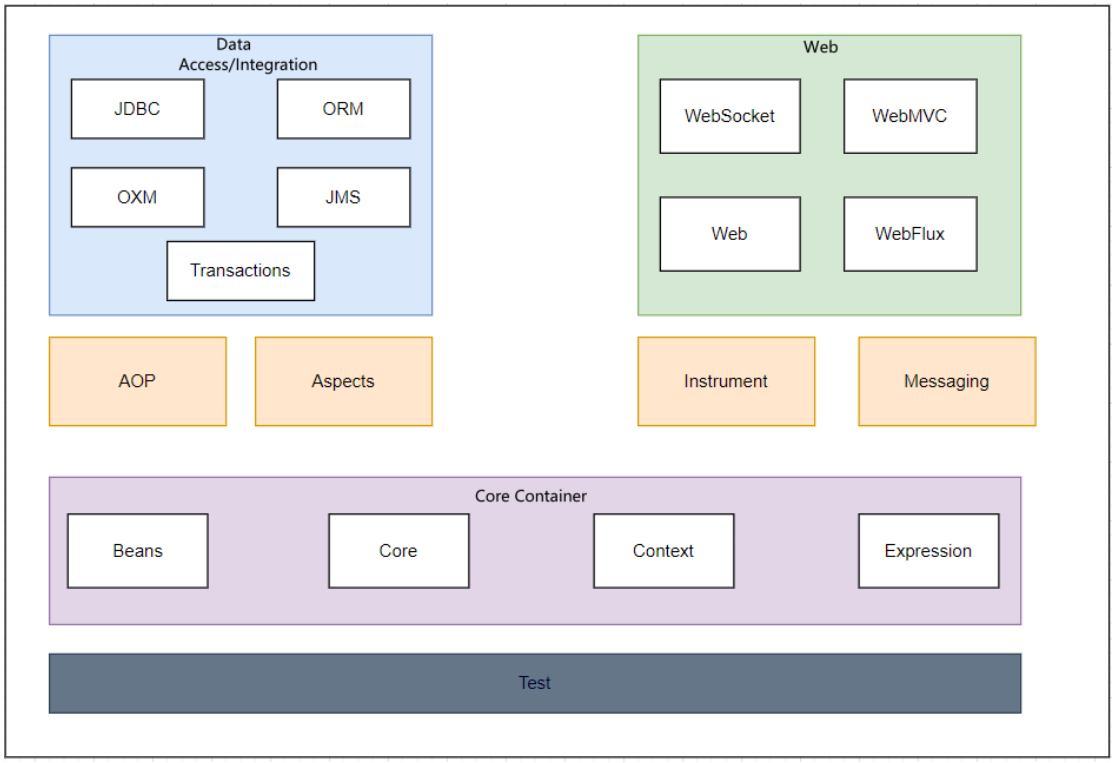

图 2.1 Spring 体系结构示意图

SpringBoot 框架介绍：

SpringBoot 是由 Pivotal 团队提供的全新框架，其设计目的是用来简化 Spring 应用的初始搭建以及开发过程。

SpringBoot 关注到开发者在进行开发时，往往对依赖版本的选择具有固定的搭配格式，并且这些依赖版本的选择还不能乱搭配，于是将所有的技术版本的常见使用方案整理了出来，以后开发者使用时直接用 SpringBoot 提供的版本方案，就不用担心冲突问题，相当于 SpringBoot 做了无数个技术版本搭配的列表，这个技术搭配列表的名字叫做 spring-boot-starter-parent，spring-boot-starter-parent 自身具有很多个版本，每个 parent 版本中包含有几百个其他技术的版本号。

同时，开发者在实际开发时，对于依赖坐标的使用往往都有一些固定的组合方式，比如使用 spring-webmvc 就一定要使用 spring-web。每次都要固定搭配着写，于是把所有的技术使用的固定搭配格式都给开发出来，以后用某个技术，就不用一次写一堆依赖，对于这样的固定技术搭配，SpringBoot 给它起了个名字叫做 starter。starter 定义了使用某种技术时对于依赖的固定搭配格式，也是一种最佳解决方案，使用 starter 可以帮助开发者减少依赖配置[4]。

基于传统的 Spring 框架开发 web 应用，我们需要做很多和业务开发无关并且只需要做一次的配置，比如管理 jar 包依赖、web.xml 维护、Dispatch-Servlet.xml 配置项、应用部署到 Web 容器、第三方组件集成到 Spring IOC 容器中的配置项维护等，而 Spring Boot 中，我们不需要再去做这些繁琐的配置，Spring Boot 已经自动帮我们完成了，这就是约定由于配置思想的体现。

Spring Boot 约定由于配置的体现有很多，比如：

1.Spring Boot Starter 启动依赖，它能帮我们管理所有 jar 包版本

2.如果当前应用依赖了 spring mvc 相关的 jar 包，那么 Spring Boot 会自动内置 Tomcat 容器来运行 web 应用，我们不需要再去单独做应用部署。

3.Spring Boot 的自动装配机制的实现中，通过扫描约定路径下的 spring.factories 文件来识别配置类，实现 Bean 的自动装配。

4.默认加载的配置文件 application.properties 等等。

2.2 SpringCloud 框架

微服务是一种软件架构风格，它将应用程序分解成小的、松耦合的服务，每个服务都执行特定的业务功能，并通过轻量级的通信机制（通常是 HTTP REST API9）与其他服务进行交互[5]。

微服务这种方案需要技术框架来落地，本项目选择的是 SpringCloud 和 SpringCloudalibaba，微服务架构示意图如图 2.2 所示：

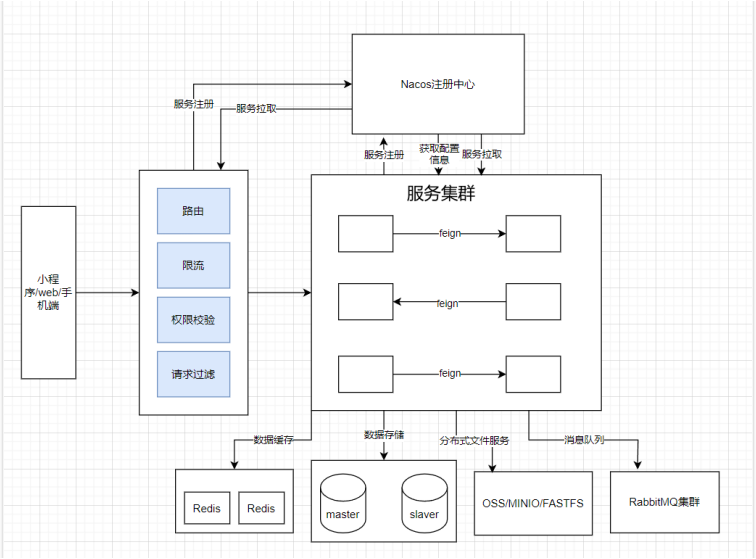

图 2.2 微服务架构示意图

Spring Cloud 是一组框架的有序集合，基于 Spring Boot 框架构建，但它并不是一个单一的框架，而是一个整合了多个微服务框架的容器。Spring Cloud 借助 Spring Boot 的开发便捷性，巧妙地简化了分布式系统基础设施的开发，包括服务发现与注册、配置中心、消息总线、负载均衡、断路器、数据监控等功能。这些功能可以通过 Spring Boot 的开发风格实现一键启动和部署[6]。

SpringCloud 并没有重复制造轮子，而是将各家公司开发的比较成熟、经得起实际考验的服务框架组合起来，通过 Spring Boot 风格进行再封装，屏蔽掉了复杂的配置和实现原理，最终给开发者留出了一套简单易懂、易部署和易维护的分布式系统开发工具包[7]。

在本商城购物系统中主要用到的 Spring Cloud 和 SpringCloudalibaba 组件有：Feign，Gateway，Nacos。

Nacos：实现服务注册和服务发现，采用心跳检测和主动询问方式保证及时更新服务列表状态；实现配置管理，可以将应用程序的配置信息存储在 Nacos 的配置中心，实现动态配置管理和灰度发布。

Gateway：作为 api 网关和独立的微服务，一切请求都必须先经过 gateway，但网关不处理业务，而是根据某种规则，把请求转发到某个微服务，这个过程叫做路由。当然路由的目标服务有多个时，还需要做负载均衡。

Feign：Feign 通过声明式的接口定义，使得开发者可以像调用本地方法一样调用远程服务，极大地简化了服务间的通信过程。

2.3 Mysql 数据库

MySQL 是一个开放源代码的关系型数据库管理系统（RDBMS），它使用结构化查询语言（SQL）进行数据库管理[8]。MySQL 由瑞典公司 MySQL AB 开发、发布和维护，现在是 Oracle 公司的产品之一。MySQL 是最流行的关系型数据库管理系统之一，在全球范围内被广泛应用于各种规模的企业。

以下是 MySQL 的一些关键特性和优点：

开源与免费：MySQL 提供了社区版，这是一个完全开源的版本，用户可以自由地使用、修改和分发。

跨平台支持：MySQL 支持多种操作系统，如 Windows、Linux、macOS、

Solaris 等等。

高性能：MySQL 在处理大量数据时具有良好的性能表现，尤其是在 Web 应用中。

可靠性：MySQL 使用 ACID 兼容的事务，确保了数据的一致性和持久性。

易用性：MySQL 安装简单，配置容易，且有丰富的文档和支持资源。

灵活的数据类型：支持多种数据类型，包括数值、日期时间、字符串、二进制等。

存储引擎：MySQL 支持多种存储引擎，比如 InnoDB、MyISAM 等，每种引擎都有其特点和适用场景。

安全性：提供安全功能，如用户权限管理、SSL 支持等，保护数据免受未授权访问。

可扩展性：可以通过分区、集群等方式来提升性能和可用性。

复制和备份：MySQL 提供了多种方式来进行数据复制和备份，以保证数据的安全性和高可用性。

2.4 Redis 缓存

redis 是一款高性能的 NOSQL 系列的非关系型数据库。

NoSQL(NoSQL = Not Only SQL)，即“不仅仅是 SQL”，是一项全新的数据库理念，泛指非关系型的数据库。

随着互联网 web2.0 网站的兴起，传统的关系数据库在应付 web2.0 网站，特别是超大规模和高并发的 SNS 类型的 web2.0 纯动态网站已经显得力不从心，暴露了很多难以克服的问题，而非关系型的数据库则由于其本身的特点得到了非常迅速的发展。NoSQL 数据库的产生就是为了解决大规模数据集合多重数据种类带来的挑战，尤其是大数据应用难题。NoSql 数据库并不是要取代关系型数据库，而是关系型数据库的补充[9]。

Redis 是用 C 语言开发的一个开源的高性能键值对(key-value)数据库，官方提供的数据是可以达到 100000+的 QPS（每秒内查询次数）。它存储的 value 类型比较丰富，它的值主要由 string（字符串），hash（哈希），list（列表），set（集合），zset（有序集合）等五种基本数据结构构成，除此之外，还支持几种复杂数据类型，比如位图（Bitmaps）、超日志（HyperLogLog）、地理空间

（Geospatial）和流（Streams）等更丰富的数据类型。位图适合高效处理大量的二进制数据，超日志用于进行基数估算并节省内存，地理空间支持存储和查询地理位置相关的数据，而流则用于处理消息队列和实时数据流。通过这些扩展的数据结构，Redis 可以应对更为复杂和多样化的应用需求[10]。

Redis 是一个基于内存的 key-value 结构数据库，将数据放在内存中，可以实现最快的对数据读取。而且 Redis 还提供了数据持久化功能，不用担心服务器重启对内存中数据的影响[11]。其次现在硬件越来越便宜的情况下，Redis 的使用也被应用得越来越多，使得它拥有很大的优势。

2.5 RabbitMQ 消息队列

RabbitMQ 是一个由 Erlang 语言编写的消息中间件，它遵循 AMQP 协议，提供了稳定可靠的消息传输服务。RabbitMQ 通过其独特的架构和丰富的功能，帮助开发者解决分布式系统中的消息传递问题，提高系统的可扩展性、可靠性和响应速度。

RabbitMQ 的架构基于生产者-消费者模型，通过队列（Queue）来实现消息的存储和转发。生产者（Producer）将消息发到队列中，而消费者（Consumer）从队列中取出并处理这些消息。RabbitMQ 还引入了交换机（Exchange）和路由键（Routing Key）等概念，以实现更加灵活和复杂的消息路由和分发机制[12]。

在本次项目中使用了 RabbitMQ,主要利用它的 ttl 和死信队列来实现延迟队列，用于解决用户提交订单却没有支付的情况。也就是说每一笔订单对于系统来说都有可能超时，当用户提交订单时，将订单信息放入一个带有 TTL 的普通队列，同时，该队列需要配置死信交换机属性，指向一个专门用于处理超时订单的死信队列。如果用户在此期间完成了支付，应用程序应该监听支付成功的事件，并立即从普通队列中删除对应的消息。如果消息在普通中达到了 TTL，而用户仍未支付，则该消息会被转移到死信队列。此时，消费者可以从 死信队列中读取消息，并执行相应的业务逻辑，比如取消订单、释放库存或通知用户订单已取消[13]。

2.6 SpringSecurity 权限校验框架

Spring Security 是 Spring 家族中的一个安全管理框架，它是用于保护基于 Spring 的应用程序的实际标准，致力于为 Java 应用程序提供身份验证和授权。与

所有 Spring 项目一样，Spring Security 的真正强大之处在于可以轻松扩展以满足自定义要求[14]。

Spring Security 一般流程为：

当用户登录时，前端将用户输入的用户名、密码信息传输到后台，后台用一个类对象将其封装起来，通常使用的是 UsernamePasswordAuthenticationToken 这个类。

程序负责验证这个类对象。验证方法是调用 Service 根据 username 从数据库中取用户信息到实体类的实例中，比较两者的密码，如果密码正确就成功登陆，同时把包含着用户的用户名、密码、所具有的权限等信息的类对象放到 SecurityContextHolder（安全上下文容器，类似 Session）中去[15]。

用户访问一个资源的时候，首先判断是否是受限资源。如果是的话还要判断当前是否未登录，没有的话就跳到登录页面。

如果用户已经登录，访问一个受限资源的时候，程序要根据 url 去数据库中取出该资源所对应的所有可以访问的角色，然后拿着当前用户的所有角色一一对比，判断用户是否可以访问（这里就是和权限相关）。SpringSecurity 基本认证流程图如图 2.3 所示：

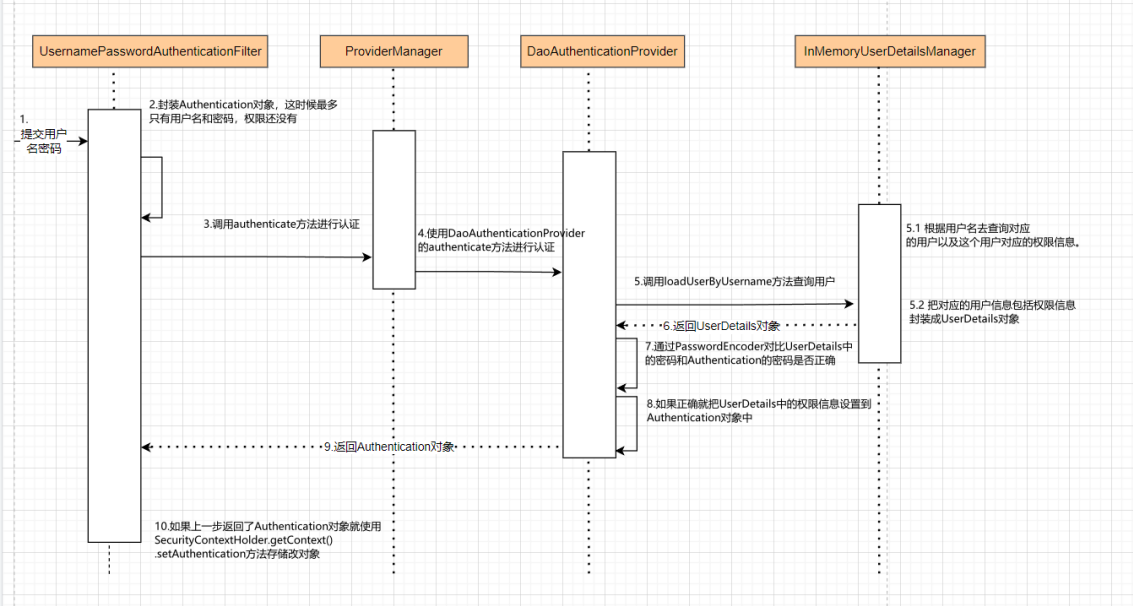

图 2.3 SpringSecurity 认证图

2.7 Minio 存储技术

分布式文件系统（Distributed File System, DFS）是一种文件系统，它使文件可以跨越多个服务器或存储设备存储和访问。DFS 通过网络将多个存储资源组合成一个统一的文件系统，使用户和应用程序可以像访问本地文件一样透明地访问远程文件[16]。

MinIO 是一个高性能的分布式对象存储系统，兼容 Amazon S3 API，旨在提供高效、可扩展、易于使用的存储解决方案。MinIO 支持大规模数据存储和高并发访问，适用于构建私有云、混合云以及容器化环境中的对象存储服务。它的设计目标是简化部署和管理，同时保证强大的性能，尤其是在存储大量非结构化数据时表现优越。

MinIO 和 FastDFS 各有优势，MinIO 在兼容性、扩展性、性能和生态系统方面表现更佳，适合云原生应用、大数据分析等场景；而 FastDFS 在处理小文件方面性能出色，适合需要海量小文件存储的场景。具体选择可以根据实际需求和使用场景来决定。

3 商城购物系统需求分析

软件生命周期是一个软件产品从概念形成到最终退役的整个过程，可分为可行性研究、需求分析、软件设计、编码、软件测试、软件维护，需求分析是设计的前提，本章对微服务购物商城的功能需求进行分析[17]。

本次商城项目采用前后端分离模式开发，用户前端使用微信小程序开发，用户后端使用微服务架构开发。而需求分析不仅要明确各个微服务的功能和服务边界，还要考虑到不同平台（如后台管理系统和微信小程序）的具体需求，以及这些需求如何映射到相应的微服务上。例如，订单服务不仅需要支持后台的查询、发货等功能，还需要处理来自微信小程序的付款请求、订单取消等操作。因此，需求分析应当细致入微，本章将根据微信小程序和后台分别进行需求分析。

3.1 后台服务端功能需求分析

后台服务端成功实现以下几个功能:订单管理、用户管理、产品管理、订单管理和门店管理，这些功能在后台管理端都得以实现。

3.1.1 订单功能需求分析

后台的订单管理模块，管理员可以看到订单的基本信息，以及订单的状态。管理员可以根据订单编号以及时间，订单状态来搜索订单，并且可以导出订单。

后台订单管理用例图如 3.1 所示：

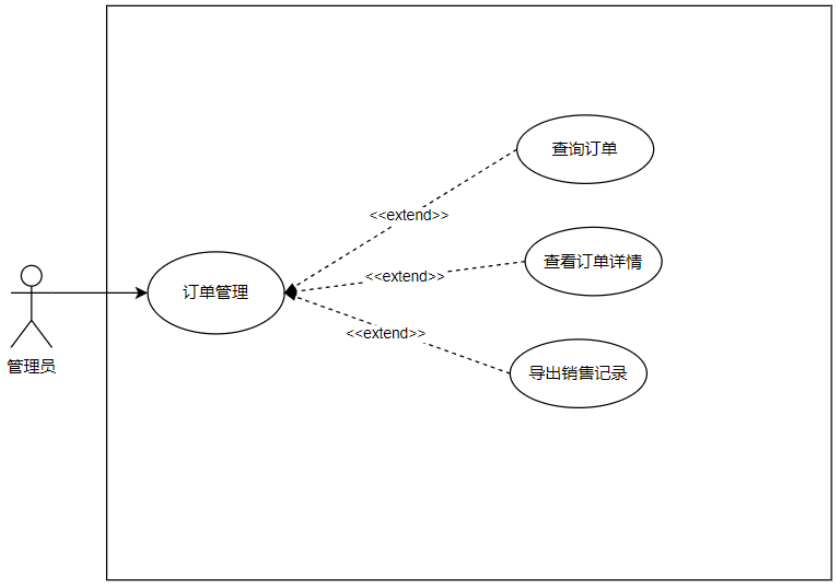

图 3.1 订单管理用例图

3.1.2 用户功能需求分析

用户管理，管理微信小程序用户的账号，管理员可以根据用户的昵称和账号状态查询用户。此外，管理员还可以禁用和删除用户账号。

用户管理用例图如图 3.2 所示。

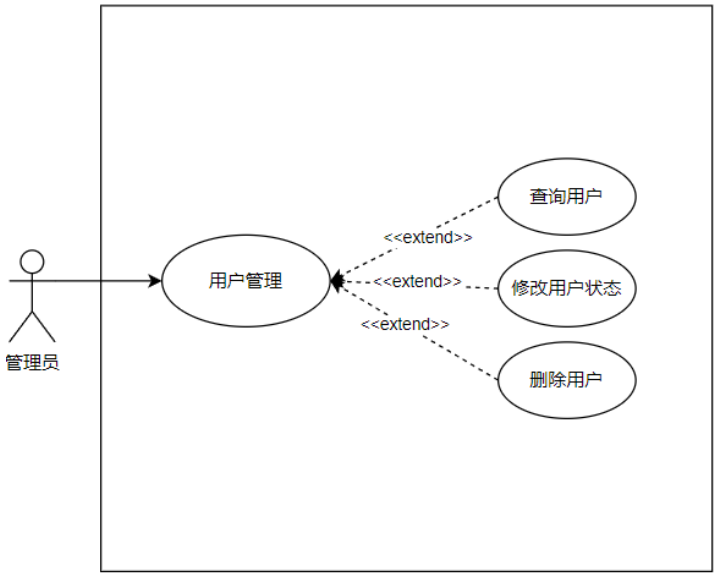

图 3.2 会员管理用例图

3.1.3 系统功能需求分析

系统服务模块，管理后台管理系统的访问权限，日志，其用例图如图 3.3 所示。

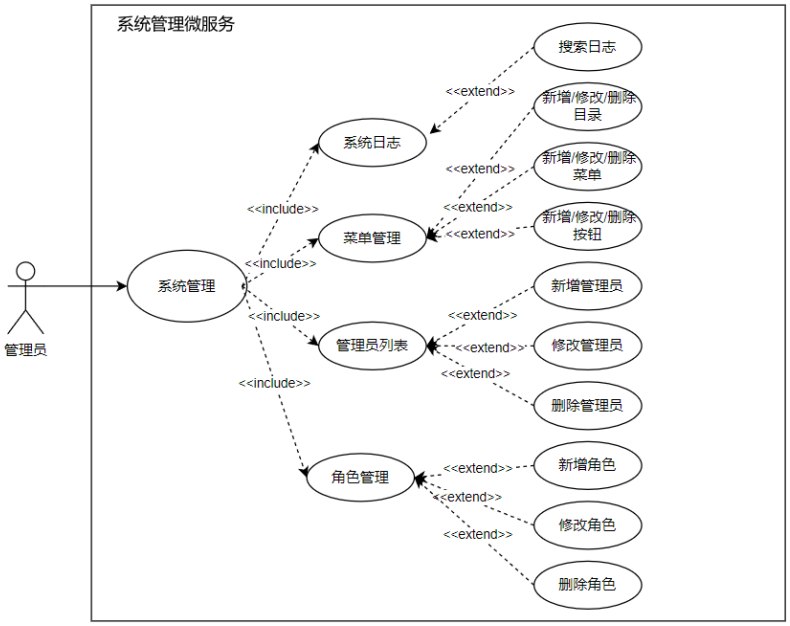

图 3.3 系统管理用例图

1.系统日志

查看接口的调用情况，比如用户操作，请求方法，请求参数，接口调用时间等。

2.菜单管理

菜单页包含后台管理系统所有的目录，菜单和按钮，拥有管理员角色的用户可以对目录，菜单以及按钮进行修改。

3.管理员列表

凡是登陆后台管理系统的用户都是管理员，但是管理员拥有不同的角色，超级管理员角色有管理系统的所有权限，而上货员角色只能对商品进行管理。

4.角色管理

角色为超级管理员的用户可以自定义角色以及可操作的权限，默认情况下只有超级管理员角色和上货员角色。

3.1.4 产品功能需求分析

产品服务是整个后台管理的核心，后台管理人员可以对商品进行管理，例如对商品进行上架下架，设置商品的规格，图片，分类等，而微信用户通过产品服务获取产品信息，了解到产品的具体细节。产品服务的用例图如图 3.4 所示。

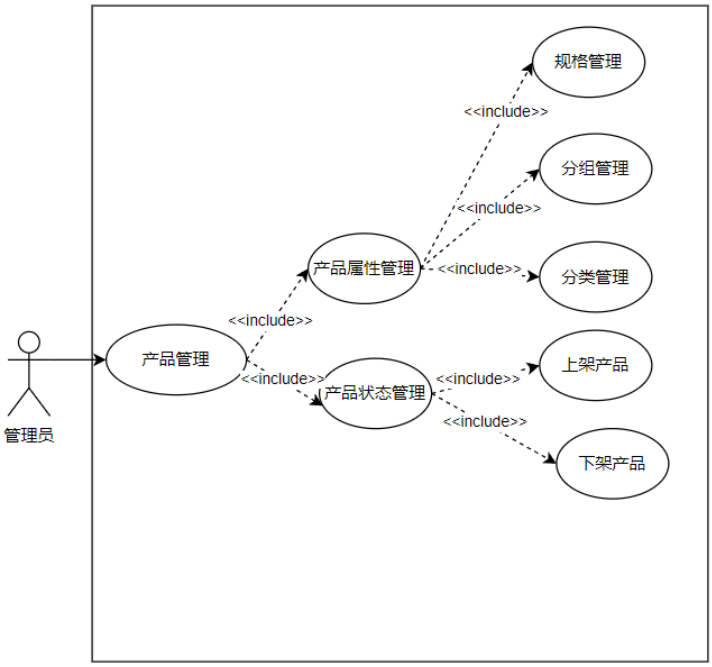

图 3.4 产品管理用例图

1.产品属性管理

产品属性管理包含三个主要功能，分别是规格管理，分组管理管理和分类管理。

规格管理，是产品的 SkU 属性，主要用于管理和维护产品的各种规格属性。管理员可以通过该模块新增、修改和删除产品规格，这些功能确保了产品信息的准确性和完整性，便于后续的产品分类和管理。

分组管理主要用于管理和组织产品标签，可以通过该模块新增、修改和删除产品标签，这些标签有助于对产品进行分类和推广，提升管理效率和用户体验。

分类管理，按照商品的共同特点来进行分组展示。

2.产品状态管理

管理员可以对产品做上架下架处理。

3.1.5 门店功能需求分析

门店服务模块主要是对收货地址，轮播图以及公告等进行管理，其用例图如图 3.5 所示。

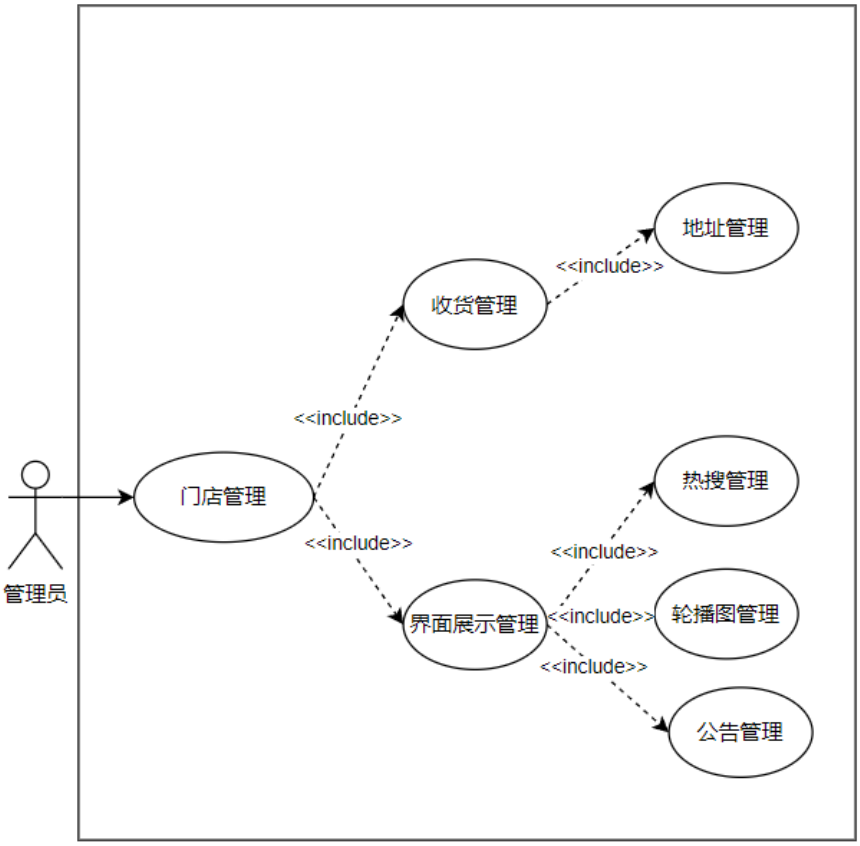

图 3.5 门店管理用例图

3.2 微信端功能需求分析

购物商城的功能通常分为两个部分,一是微信端,二是后台管理端,前者主要功能包括用户的商品展示、商品搜索、商品交易、等功能,以下主要是针对其功能需求进行讨论。

3.2.1 前台门户功能需求分析

前台门户作为用户接触商城的第一触点，其布局直接影响浏览效率与购买决策。所以门户网站的设计对于电商系统来说至关重要。本文微服务商城系统门户网站主体是商品分类分组信息、主流热门商品展示、轮播图展示、公告展示和导航栏,其用例图如图 3.6 所示。

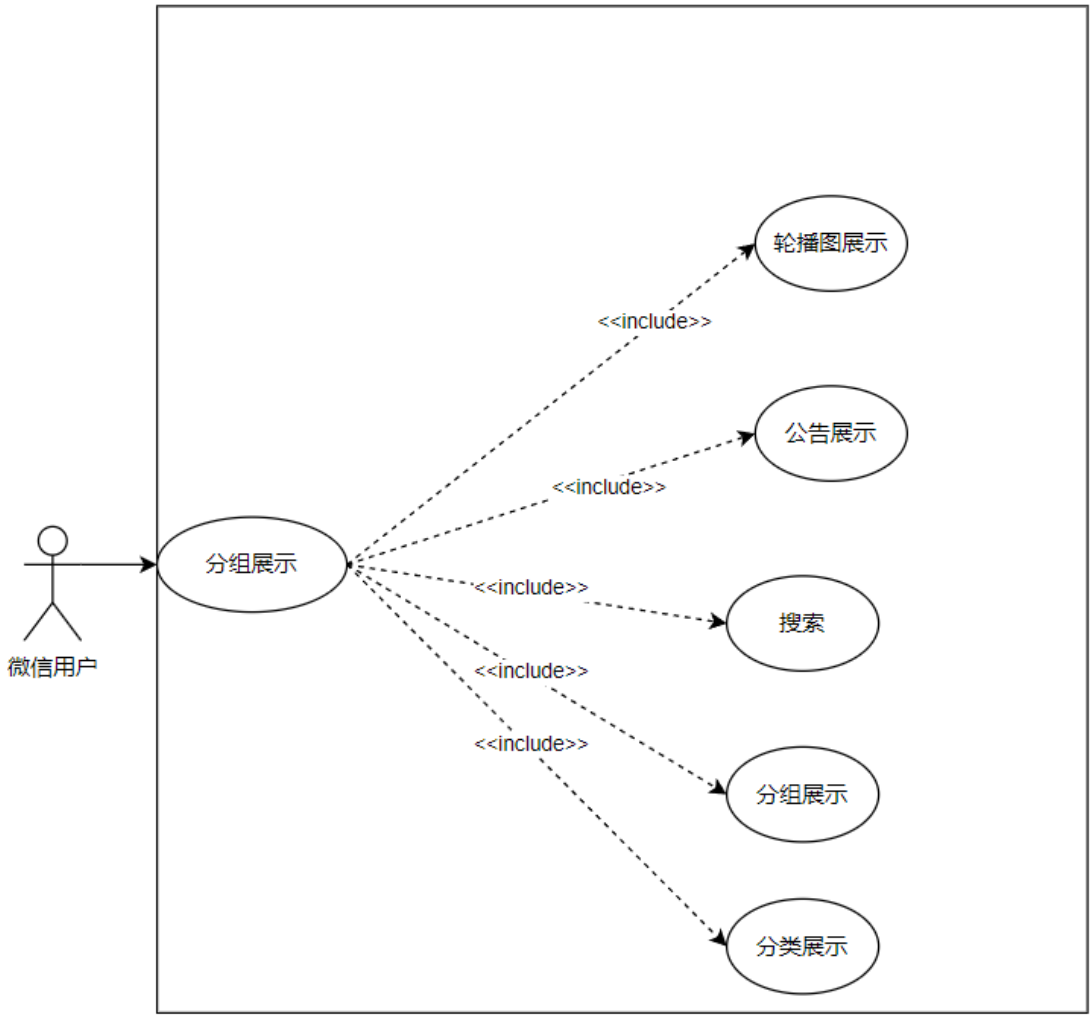

图 3.6 前台门户功能用例图

顶部轮播图需动态关联爆款或活动商品，通过视觉焦点快速吸引用户点击；商品展示区按分类标签分组陈列（如“热门推荐”“新品上市”）。搜索框需支持模糊查询与历史记录。公告区域则展示通知。

3.2.2 用户中心功能需求分析

用户中心作为购物流程的关键点，需要确保用户下单后能实时的追踪订单状态、管理常用地址以及并快速找回心仪商品。订单查看功能需覆盖全流程，帮助用户掌握购物进展与商品明细；收货地址管理需支持省市区联动选择和默认地址设置，避免重复填写提升下单的效率；收藏功能则需直观呈现商品主图与价格。其用例图如图 3.7 所示。

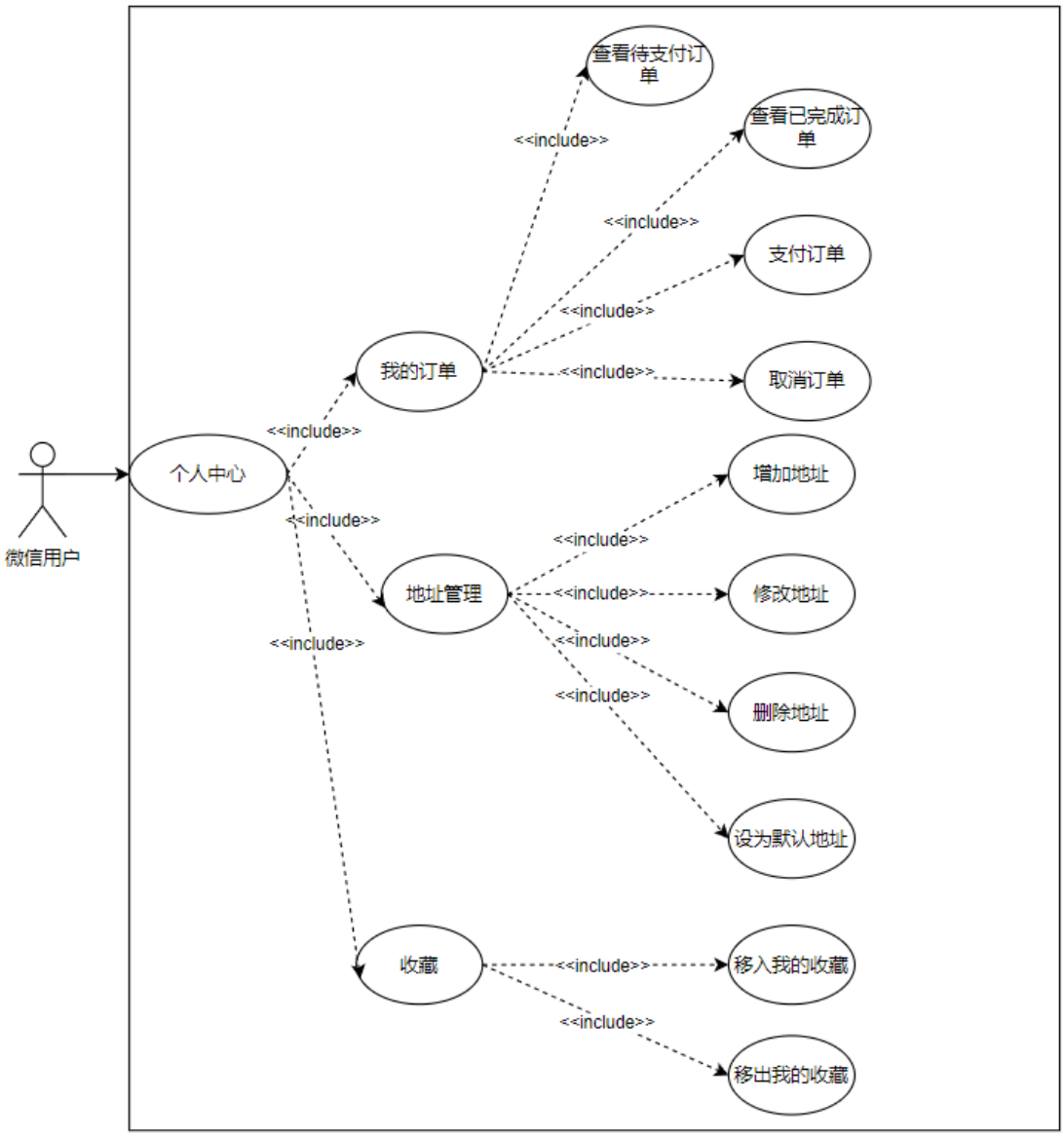

图 3.7 用户中心功能用例图

3.2.3 购物车需求分析

商城系统的购物车功能是电商平台的重要组成部分，主要负责存储用户拟选购商品的信息，并提供灵活的操作功能。用户可以随时将感兴趣的商品加入购物车，方便地调整商品数量，或移除不再需要的商品。购物车功能还需实时计算选中商品的总金额，确保用户在结算前能够清楚了解价格信息。结算流程应尽可能简化，减少跳转层级，提升用户体验，支持批量勾选商品并生成待支付订单，优化结算效率。该功能的实现不仅提升了系统的灵活性，还增强了用户购物过程中的便捷性与流畅性。其用例图如图 3.8 所示。

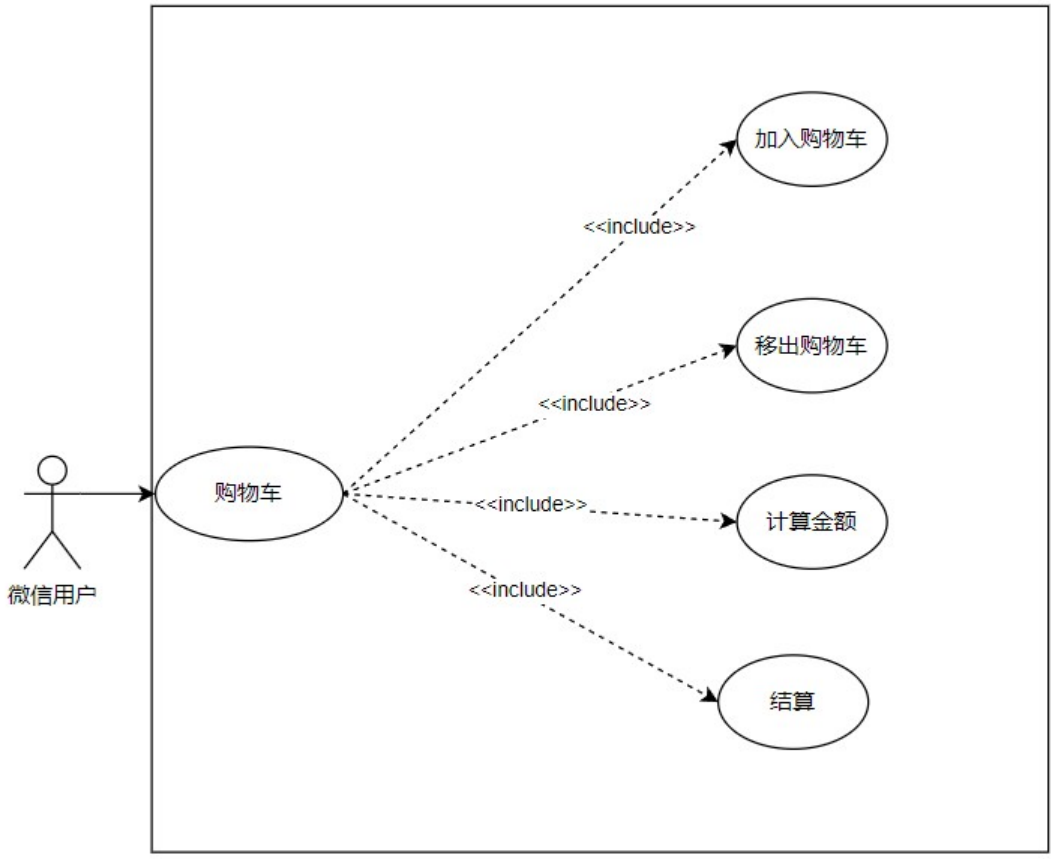

图 3.8 购物车功能用例图

3.2.4 订单功能需求分析

订单模块作为交易流程的核心环节，需贯穿用户从下单到履约的全链路管理。订单详情页需展示商品明细、支付金额、地址信息等关键信息，确保用户可以清晰查看订单内容。支付环节需对接第三方支付通道，并嵌入倒计时机制，若支付超时未完成，系统将自动释放库存并关闭订单，避免资源闲置与误占，确保平台资源的高效利用。用户可手动清理已完成或已取消的订单记录，但系统需保留订单的基础数据，便于历史追溯与数据分析。通过这一机制，平台能够提供流畅、稳定的交易体验，并支持日后的业务优化。其用例图如图 3.9 所示。

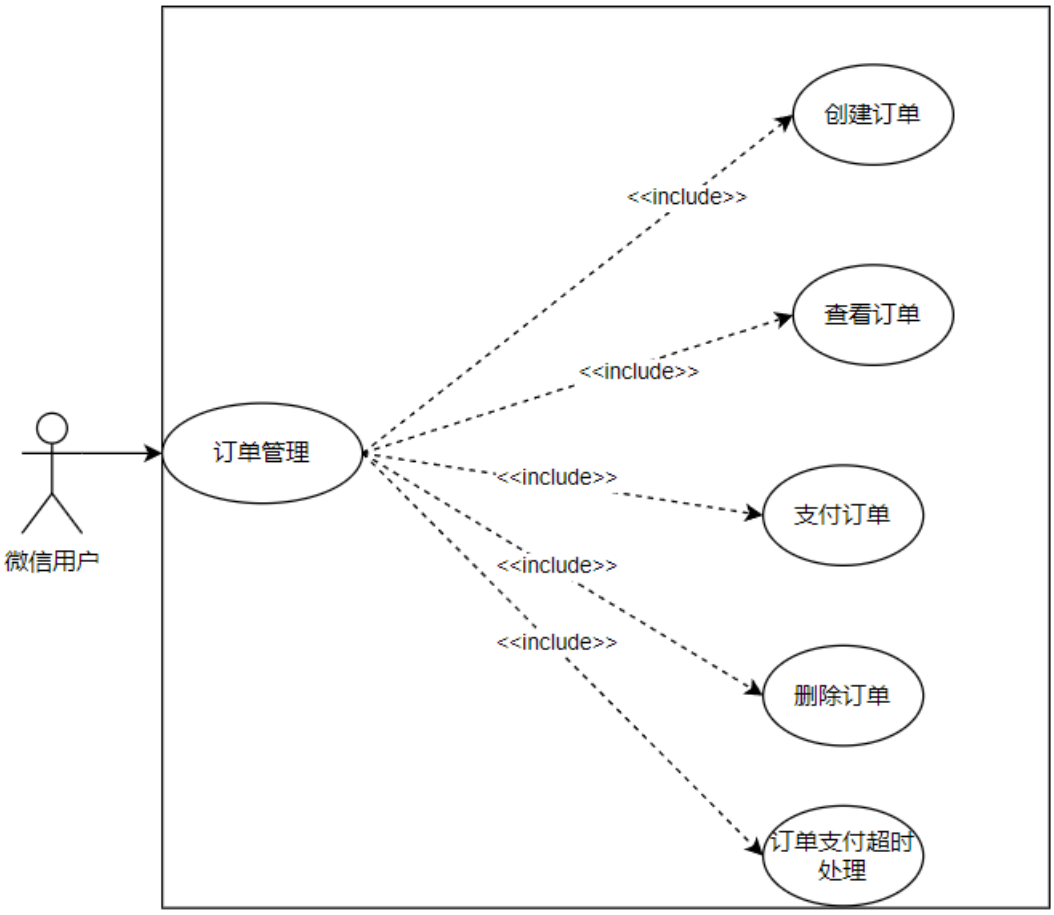

图 3.9 订单功能用例图

4 数据库设计

4.1 Linux 里安装数据库

在官网下载 mysql-8.0.32-1.el7.x86_64.rpm，上传到 Linux 虚拟机，

通过 scp 命令上传到 Linux，命令为：

scp 文件本地路径 服务器用户名@服务器 ip 地址:上传目标的绝对路径。

解压 mysql 安装包，命令为：

tar -xvf mysql-8.0.32-1.el7.x86_64.rpm-bundle.tar

安装所有的 rpm 包，命令为：

rpm -ivh mysql-community-common-8.0.32-1.el7.x86_64.

rpm --nodeps --forcerpm -ivh mysql-community-libs-8.0.32-1.el7.x86_64.

rpm --nodeps --forcerpm -ivh mysql-community-client-8.0.32-1.el7.x86_64.

rpm --nodeps --forcerpm -ivh mysql-community-server-8.0.32-1.el7.x86_64.rpm --nodeps –force

初始化数据库：

在 mysql-8.0.32 目录下，命令：mysqld --initialize

修改 mysql 权限：

在 mysql-8.0.32 目录下，命令：chown mysql:mysql /var/lib/mysql -R

启动 MySQL 数据库服务：

在 mysql-8.0.32 目录下，命令：systemctl start mysqld.service

查看安装 MySQL 初始化密码：

命令：cat /var/log/mysqld.log | grep password

使用初始化密码登陆 mysql,命令：mysql -uroot -p

修改 MySQL 密码：

命令：alter user 'root'@'localhost' identified by '123456';

使用新密码登陆 mysql，结果如图 4.1 所示：

```
[root@localhost ~]# mysql -uroot -p123456
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 222
Server version: 8.0.32 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

图 4.1 mysql 登陆成功图

4.2 数据库逻辑模型设计

数据库设计中最常采用的模型为实体（Entity）关系（Relationship）模型，简称 ER 模型。其核心思想是将现实世界中的复杂数据表示为一组实体，并描述这些实体之间的关系。

ER 图通常用于构建概念模型，其只关注实体和关系，不体现最终建表所需的各种细节信息。

然后，在逻辑模型阶段，ER 图中的概念被更加具体化为适合实现的数据结构，会包含更多的细节信息，例如实体的属性、用于关联两个实体的字段等等；最后，在物理模型阶段，这些逻辑结构被转化为具体的数据库架构，考虑到特定 DBMS 的特点和性能要求。

这里将基于逻辑概念模型来介绍各个数据库之间的关系。

4.2.1 系统业务表

系统业务表包括：系统用户表、系统角色表、系统权限表、系统用户与角色关系表、系统角色与权限关系表。

每个用户都拥有自己的用户 id,用户名，密码等基本信息，一个用户可以有多种角色，通过中间表建立联系。每个角色又有不同的权限，每种权限可以执行的操作各不相同，通过用户-角色-权限-资源这种模型来控制用户授权。

该业务所涉及的 5 张表的逻辑模型如下图 4.2 所示。

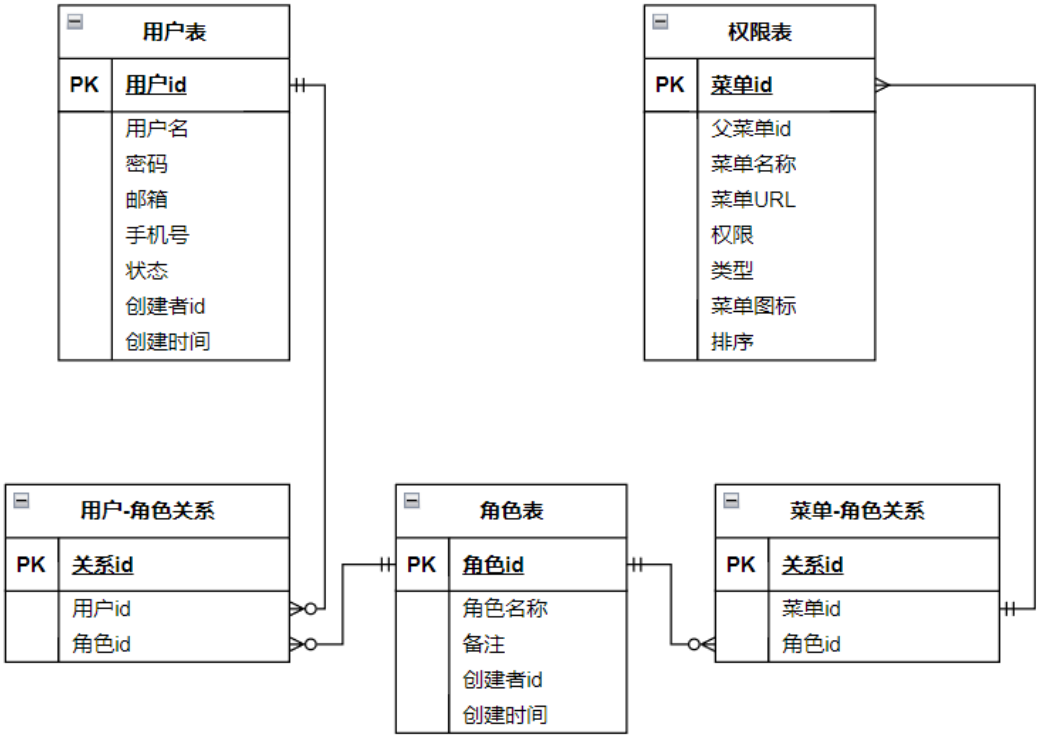

图 4.2 系统业务表逻辑模型

4.2.2 商品业务表

商品业务表包括：商品类目表、商品分组标签表、商品与分组标签关系表、商品属性表、商品属性值表、商品评论表、商品表、商品 sku 表。

商品按照多级目录的方式进行管理，商品可以根据各自的特性放到不同的类目下，所以商品多级目录与商品之间是一对多的关系，每个多级目录拥有多个商品，每个商品又拥有多个 sku。商品分组将这些商品管理和组织起来，1 个商品可以有多个属性分组，是 1 对多的关系。商品属性表则定义了商品的不同特性，如颜色、尺寸等，并且每个属性可能有多个对应的属性值。商品评论表记录了用户对商品的评价信息，提供了商品的用户反馈。商品与分组标签的关系表则管理商品和标签之间的多对多关系，便于对商品进行分类和检索。

该业务所涉及的 8 张表的逻辑模型如下图 4.3 所示。

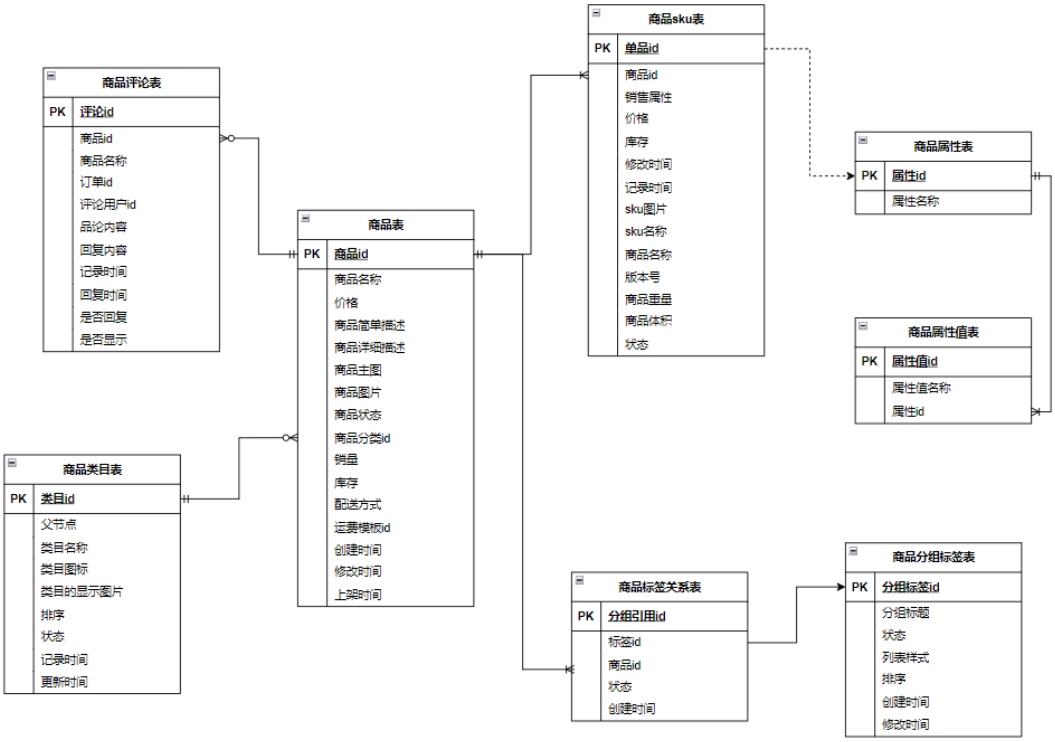

图 4.3 商品业务表逻辑模型

4.2.3 门店业务表

门店业务表包括：公告表、轮播图表、地区表。

三张表的逻辑模型图如图 4.4 所示。

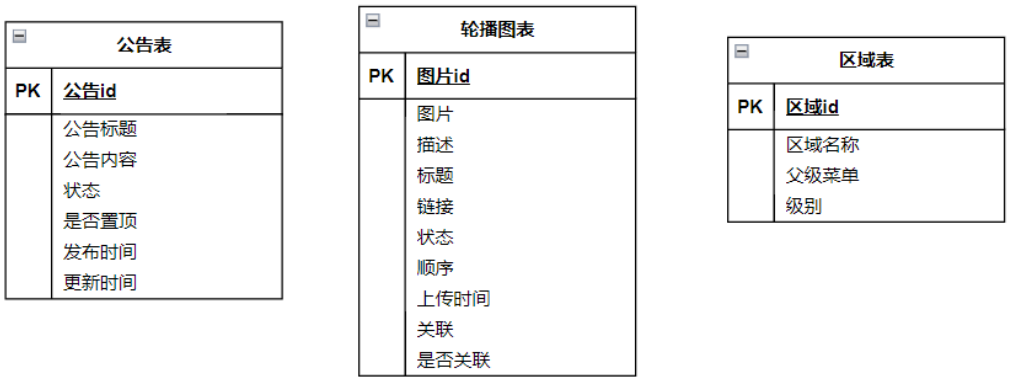

图 4.4 门店业务表逻辑模型

4.2.4 会员业务表

会员业务表包括：会员用户表、会员用户收藏记录表、会员用户收货地址表。

每个用户可以设置多个收货地址，因此用户表和地址表是 1 对多的关系，同样每个用户可以收藏多个商品，所以用户和商品之间也是 1 对多的关系。

该业务所涉及的 4 张表的关系如下图 4.5 所示。

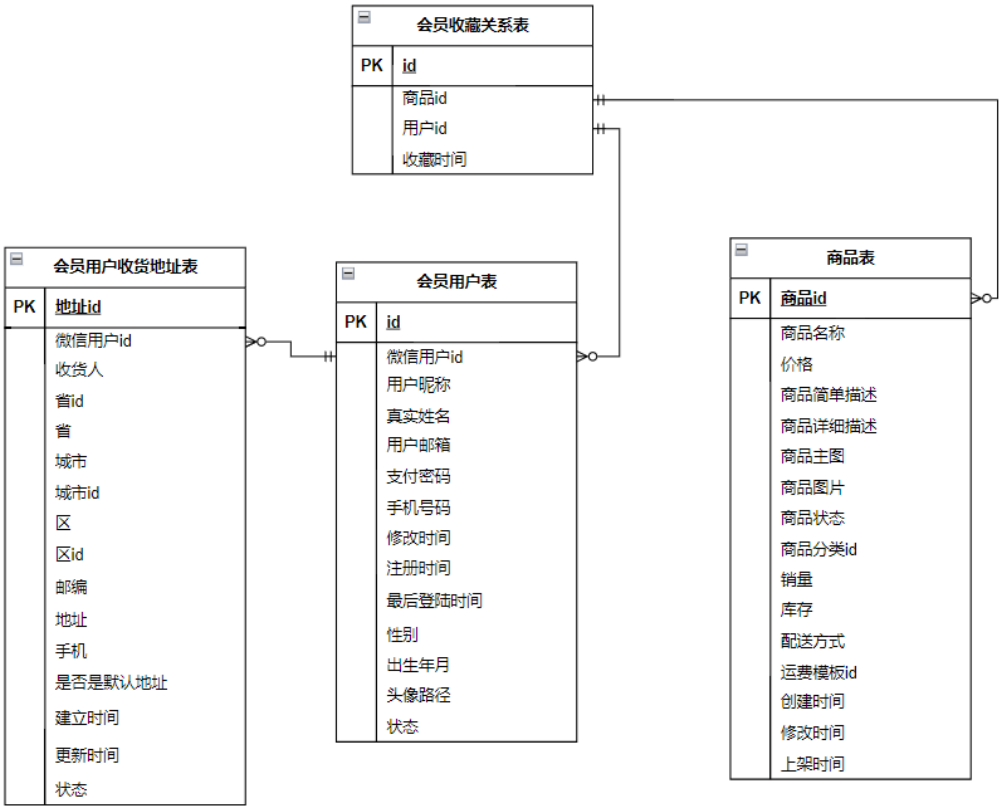

图 4.5 会员业务逻辑模型

4.2.5 购物车业务表

购物车业务表包括：购物车表。

购物车和产品表，产品属性表，用户表所关联，其逻辑模型图如图 4.6 所示。

图 4.6 购物车业务逻辑模型图

4.2.6 订单业务表

订单业务表包括：订单表、订单商品条目表。订单表记录了每个订单的基本信息，如订单编号、用户信息、订单状态、支付状态等；而订单商品条目表则记录了每个订单中包含的商品信息，如商品 ID、数量、单价、总金额等。每个订单表对应 1 个或多个订单条目标，订单条目表通过订单 ID 与订单表进行关联，以便精确管理每个订单的商品详情。其逻辑模型如图 4.7 所示。

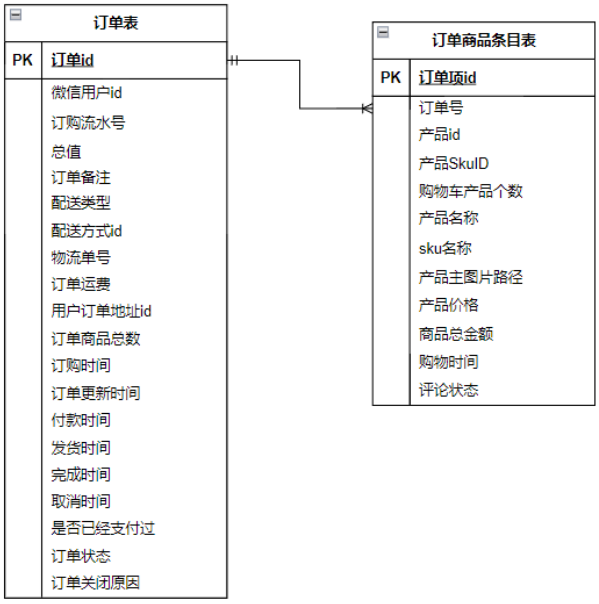

图 4.7 订单业务逻辑模型图

5 商城购物系统设计

软件设计是软件开发中的关键环节，它决定着软件开发的具体实现，下面基于上诉的需求分析，对微服务商城系统进行概要设计。

5.1 系统架构设计

微服务商城项目是基于 MVC 三层架构的基础上进行设计的，分为视图层，服务层，数据层，视图层进行页面的展示，后台管理系统和微信小程序的页面展示都是由视图层来负责的，视图层向服务层发起请求，服务层对来自视图层的请求进行处理，根据不同的请求来完成不同的逻辑处理，和数据层进行数据交互后，返回数据给表现层用于渲染页面。其总体架构设计如图 5.1 所示。

在 MVC 三层结构的基础上，根据实际需求整合到微服务架构。微服务架构严格控制服务拆分的粒度，每个服务遵循单一职责的原则，根据业务的不同拆分成不同的微服务，每个微服务之间仍然保持架构的稳定，修改其中的微服务不会影响到其他服务的进行，拆分后的每个子服务依然遵循 MVC 的架构，每个微服务之间通过 Feign 进行远程调用[18]。

商城购物系统根据业务功能划分为 6 个子服务，分别是订单服务、用户服务、系统服务、门店服务、购物车服务、产品服务、门店服务、搜索服务，每个微服务模块之间相互独立，对外提供接口，微服务之间通过 Feign 进行远程调用。在这之外，还具有处理数据的公共模块，公共模块把微服务都需要的公共提取出来，需要使用到的模块直接引入公共模块即可使用里面包含的工具。

在视图层请求到达业务之前，需要经过 1 个网关服务，网关是所有微服务的统一入口。网关负责网络路由，有了网关，android、ios、pc 前端、微信小程序、H5 等等，不用去关心后端有多少个服务，只需要有一个网关，所有请求都经过网关，网关会根据请求中的一些特征，将请求转发给后端的各个服务[19]。而且有一个网关之后，还有很多好处，比如可以做统一的降级、限流、认证授权、安全等等。

在经过网关后，请求并不会直接到业务层，而是需要经过 Spring Security 的认证授权，Spring Security 是一个功能强大且高度可定制的身份验证和访问控制框架，主要的功能就是实现用户的认证和授权，验证当前访问系统的是不是本系统的用户，并且要确认具体是哪个用户，经过认证后判断当前用户是否有权限进

行某个操作。在传统的微服务里，一般是使用过滤器或拦截器来进行安全功能的实现，但是使用 Spring Security，不仅简化了安全功能的实现，而且增强了应用程序的安全级别，同时提供了极大的灵活性以适应不同的需求[20]。

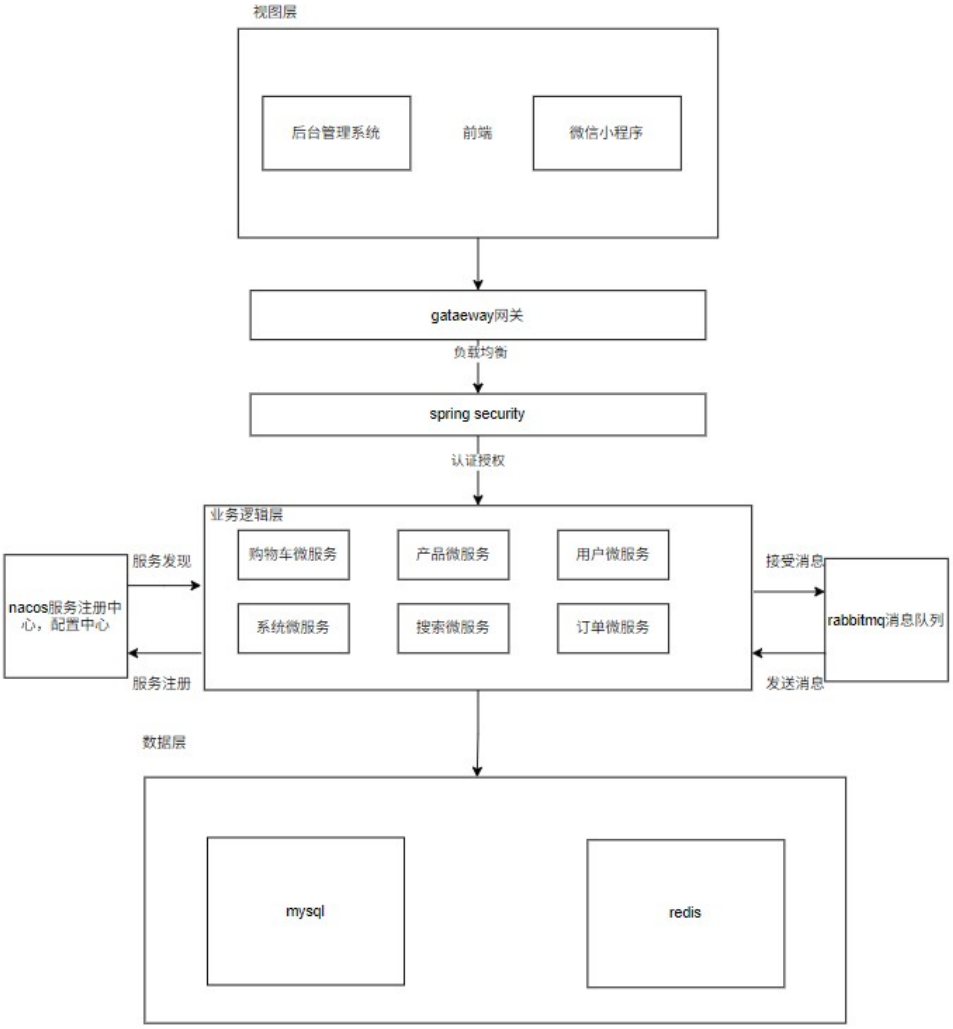

图 5.1 微服务商城总体架构设计图

5.2 系统功能模块设计

5.2.1 网关微服务设计

网关微服务模块主要提供验证请求合法性的功能。

在微信端和后台管理端发起请求时，需要一个统一的入口才能来到服务端，这个统一的入口就是网关，微服务网关是项目的统一入口，可以根据用户的请求进行路由转发，将用户的请求解析到最终的微服务中去。微服务商城项目中使用动态路由，即根据微服务的名称，进行服务发现并转发配置。

根据需求分析可以知，不是所有的请求都可以直接到达服务端，微服务网关需要验证项目所有请求的合法性，用户请求头中必须携带由商城项目的认证授权服务模块颁发的 token 令牌（即用户身份）,只要用户请求携带了合法的 token,便可以放行。但是如果所有请求都需求验证 token 的话，那就没有请求能成功到达微服务颁发 token 了，所以对于登陆请求，需要设置一个“白名单”,只要请求路径符合白名单的规则，则直接放行，进而颁发 token。

登陆成功后由登陆成功处理器颁发 token，将 token 存放到 redis 中，标识用户处于登陆状态。后续所有的请求都会携带该 token。

整个网关服务的流程图如图 5.2 所示：

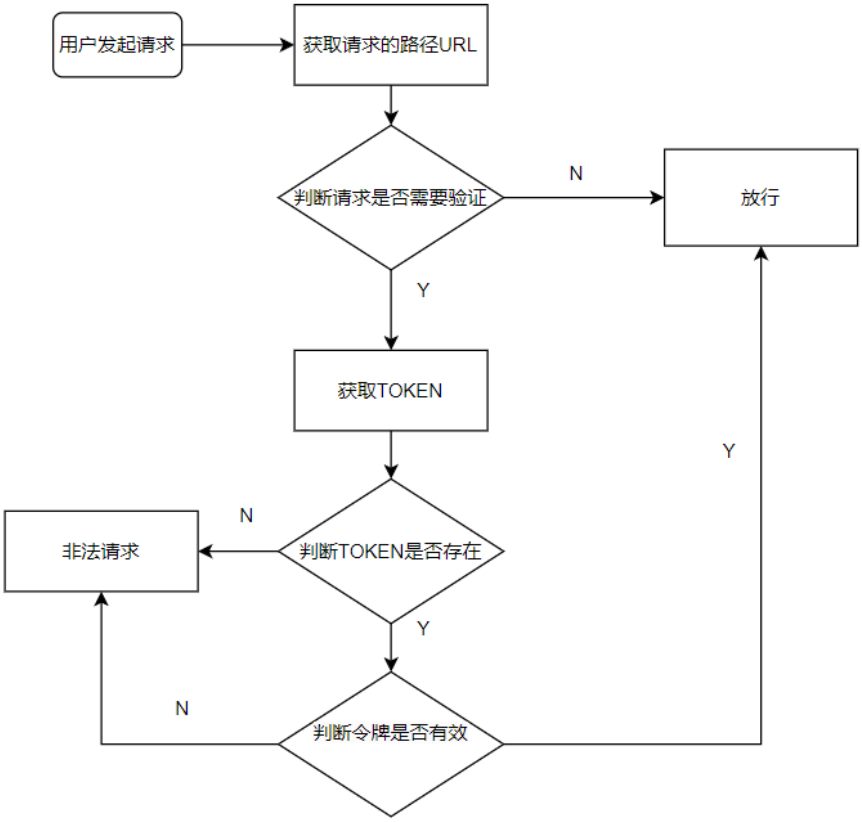

图 5.2 网关服务流程图

5.2.2 认证授权微服务设计

认证授权微服务模块主要提供认证授权服务，为整个系统提供统一的用户认证和授权管理。

由网关微服务设计可知，网关放行请求后，由认证授权服务进行校验，由于该购物商城分为后台管理系统和微信小程序，需要设计两套不同的认证流程，可以根据请求头中携带的登录类型来判断登录来源，具体流程如下：

1.后台用户登陆

用户打开后台管理系统，输入用户名和密码后，发起登陆请求，网关放行后，由 SpringSecurity 框架进行认证授权，根据用户输入的用户名和密码去数据库里查询登陆用户和对应的资源权限，如果查询到用户数据则登陆成功，由登录成功处理器颁发 token,且将 token 当作 key，认证用户对象的信息作为当前 value 存放到 redis 中，后续对后台管理系统的操作都会从 redis 里查看是否存在该 token。登陆流程图如 5.3 所示：

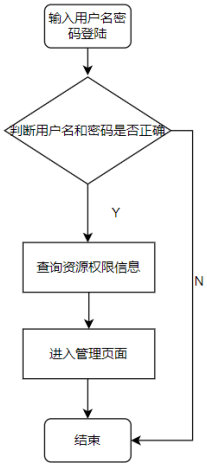

图 5.3 后台登陆流程图

2.微信用户登陆分析

进入微信小程序会自动发送登陆请求，携带临时校验码和默认密码来换取 sessionkey 和 openId，如果根据 openId 去数据库里可以查询到用户数据，则登陆成功，否则直接注册用户。

登陆流程图如图 5.4 所示：

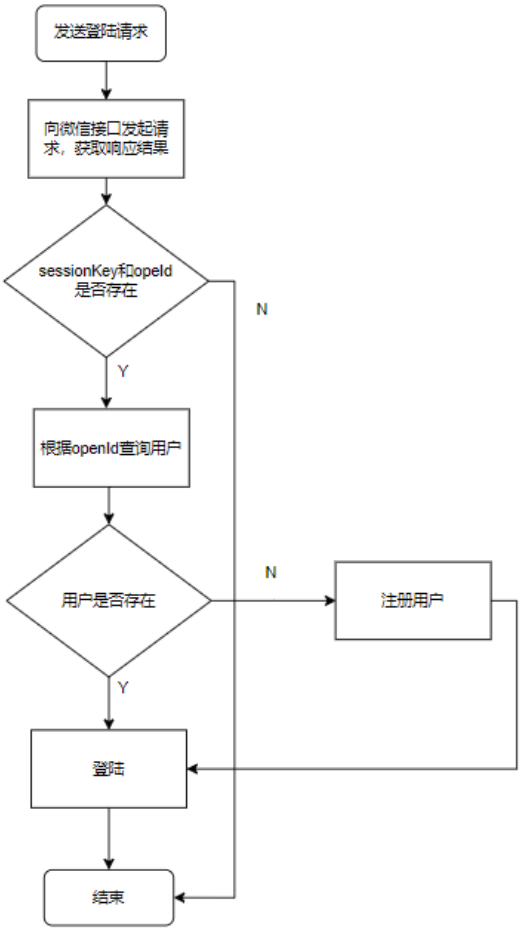

图 5.4 微信登陆流程图

5.2.3 订单微服务设计

订单微服务主要负责订单相关的业务，比如订单创建，订单查询，订单删除三个功能，其业务结构图如 5.5 所示。

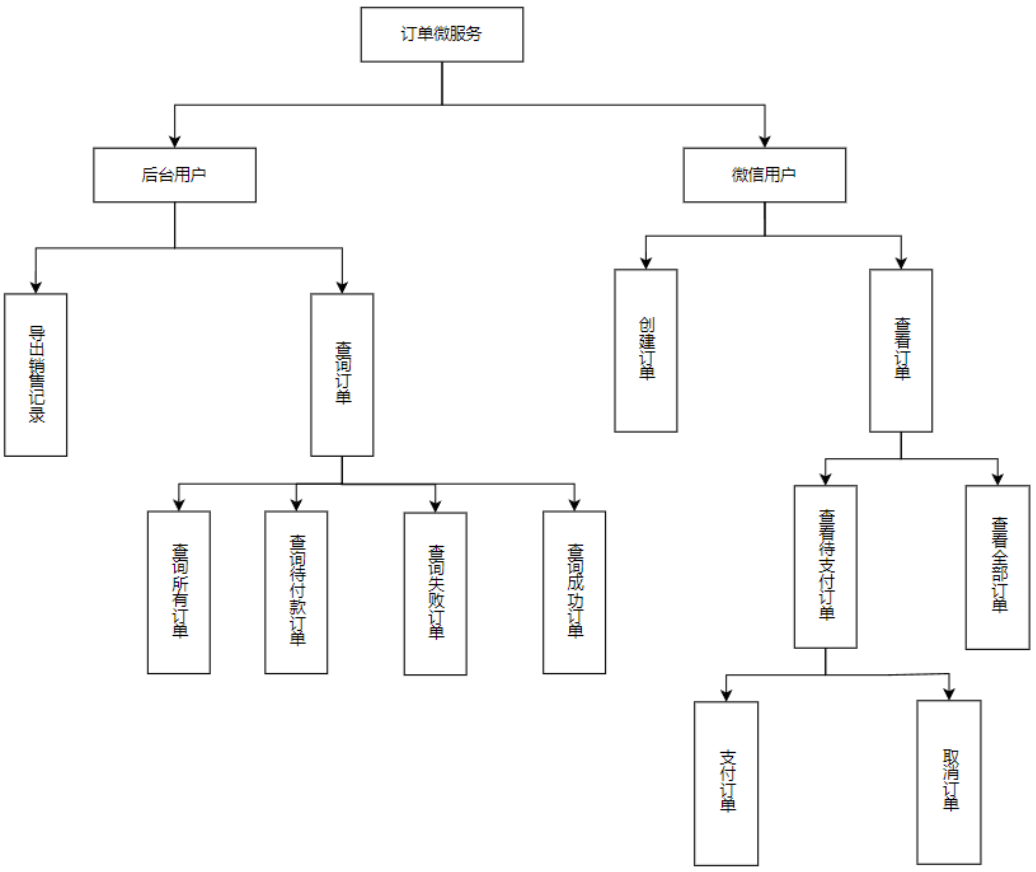

图 5.5 订单业务结构图

1.创建订单

微信用户进入商品详情页，选择立即购买，将会进入订单详情页，订单详情页展示商品详情，订单的金额，地址信息等，用户点击提交订单即可创建订单，此时订单处于未支付状态。

2.查看待付款订单

在微信用户查看待付款订单时，除了显示当前订单的信息外，还会提供取消订单和付款两个操作，用户选择取消当前订单时，会将当前用户订单状态置为已取消的状态。当微信用户在使用付款后，系统会把当前用户信息和订单相关的数据发送给订单微服务进行支付操作，支付成功后，会将订单状态置为已支付。

3.查询订单

后台用户可以根据订单的不同状态来查询订单，查询出的订单包含订单编号、商品信息和支付金额等。

4.导出销售记录

在订单页面，后台管理用户选择导出销售记录时，可以导出所有交易成功的订单。

5.2.4 用户微服务设计

基于需求分析与实际业务的考量，本系统将用户个人中心功能独立设计为专用微服务模块，重点处理用户收货地址、收藏以及用户管理等核心功能。鉴于订单业务涉及复杂的管理流程和权限控制，已遵循领域驱动设计原则将其解耦为独立微服务模块（前文已详细阐述其架构设计）。各微服务间的功能划分与交互关系通过图 5.6 所示：

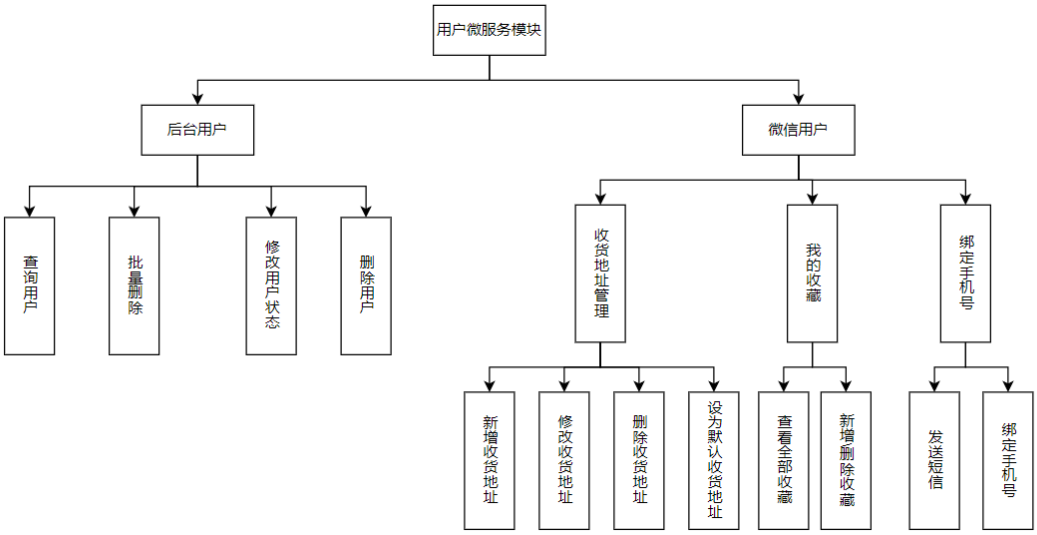

图 5.6 用户业务结构图

1.后台管理用户进入用户管理页面，列表显示所有微信用户的个人信息，包括用户昵称、用户头像、状态，并且提供了查询用户、修改用户、删除以及批量删除用户功能，其中修改只允许修改用户的状态，当用户状态为禁用时，用户将无法登陆。

2.在收货地址页面，列表显示所有的收货地址，并提供了新增收货地址、修改收货地址、删除收货地址、设置为默认收货地址功能。其中默认收货地址会在每次购物时默认展示在地址栏中，默认收货地址只能有一个，可以根据需要手动设置默认收货地址。

3.在收藏页面中，页面展示当前用户收藏的商品，用户可以点击列表中的商品跳转到商品详情页面。

4.用户在登陆后需要绑定手机号，需要绑定手机号才可以进行购物。在用户选择去绑定时，需要输入正确的手机号，先去获取验证码，只有手机号和验证码都正确才绑定成功

5.2.5 购物车微服务设计

购物车服务核心功能围绕商品管理。用户可以将商品添加到购物车中，实现商品的临时保存以便后续查看或购买。此外，用户可以对购物车中商品的数量进行调整，并且允许计算购物车内已选商品总金额。用户还可以从购物车中删除不再需要的商品，最后，用户可以在购物车页面进行结算。其业务结构图如 5.7 所示：

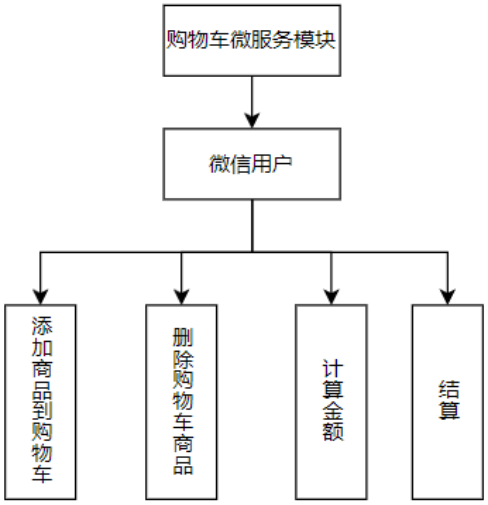

图 5.7 购物车业务结构图

在商品页面点击加入购物车，先检查你的购物车里是否存在该商品，如果存在，则自动增加数量；如果是第一次添加，就会直接放进购物车里。进入购物车后，可以选择增加或减少按钮随时调整商品数量，选择删除可以删掉不想要的商品。勾选商品时，底部会算出这些商品的总价。最后选中要买的商品，提交后生成预支付订单，同时清空已经结算的商品。

5.2.6 系统微服务设计

系统管理微服务主要对后台管理系统进行管理和维护，包括系统日志、角色管理、管理员管理、菜单管理等，系统服务结构图如图 5.8 所示：

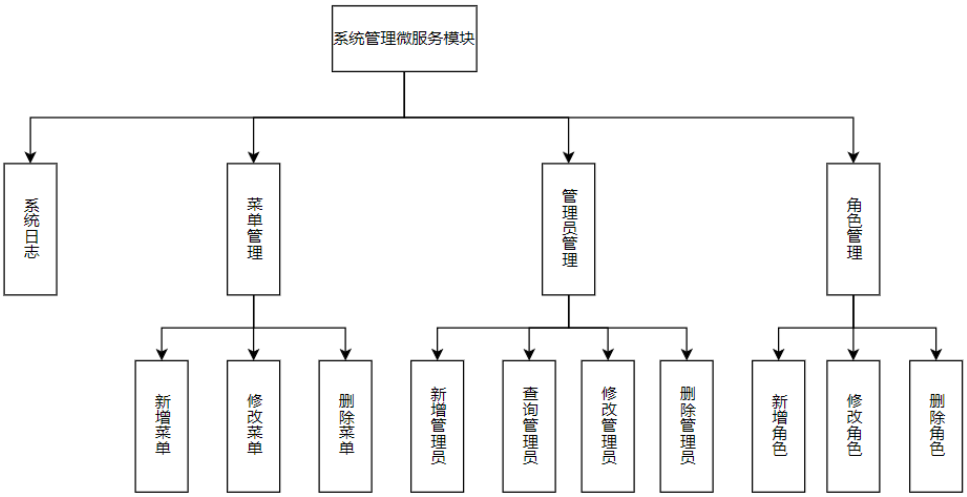

图 5.8 系统业务结构图

系统管理微服务相当于整个商城后台的"控制中枢"，主要负责搭建灵活可配置的管理体系。具体来说分为一下四个方面：

1.在系统日志页面，用户可以查看整个系统接口的调用情况，包括调用接口的用户 id、用户执行的操作、请求方法路径、请求参数、ip 地址等，系统日志可以记录系统运行过程中的各类操作，方便后续追踪和问题排查。

2.菜单管理页面展示后台管理系统的导航结构，比如目录、菜单、按钮，通过菜单管理模块可以动态控制后台导航结构，包括新增、修改和删除菜单项，以适应不断变化的业务需求。

3.在管理员列表页面，展示后台管理系统所有用户的信息，包括用户名、邮箱、手机号、状态等。并且支持账号的新增、修改和删除操作，便于管理员团队的调整和管理。

4.在角色管理页面，默认展示所有的角色信息外，并且提供了创建、修改和删除角色的按钮。在创建和修改角色功能里，最重要的是设置该角色拥有的权限范围，每个角色都对应一组特定的权限，每个用户都拥有一个特定的角色，因此每个用户有自己访问的功能范围，这样可以确保每个用户只能执行其权限允许的操作。

5.2.7 搜索微服务设计

搜索服务核心功能聚焦于商品检索与智能排序，支持通过分类 ID 精准筛选商品列表、基于标签名称匹配关联商品集合以及根据商品关键词实现全文模糊搜索其业务结构图如 5.9 所示。

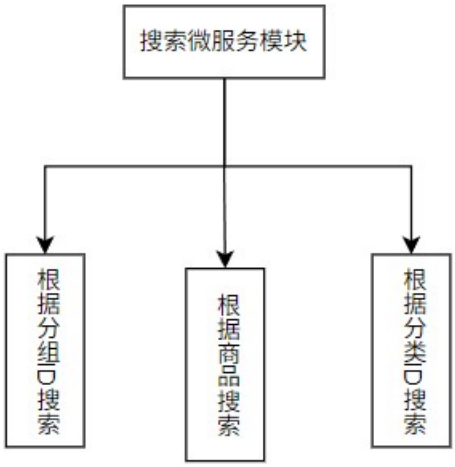

图 5.9 搜索业务结构图

1.在微信小程序首页采用动态分组标签（如“限时秒杀”“新品首发”“爆款榜单”）展示商品，每个标签模块右侧设置’更多’悬浮按钮，点击后跳转至全屏商品聚合页，会展示该分组标签下的所有商品 。

2.分类页采用二级树状结构布局，左侧垂直导航栏展示家电、服饰、食品等一级类目，点击后右侧主区域分栏呈现具体商品。

3.在首页顶部配备搜索框，用户可以根据商品名称模糊查询，查询出的商品按照综合排序展示，用户可以根据需要选择价格排序和销量排序。

5.2.8 产品微服务设计

产品微服务模块核心功能主要聚焦于商品全生命周期管理，通过分组管理实现首页商品标签化动态展示，支持创建、编辑或删除分组标签；分类管理基于多层级商品分类体系来构建结构化展示逻辑，管理员可动态调整分类树节点、设置分类属性并绑定商品集合，确保分类页面按预设规则精准呈现商品；规格管理维护商品基础属性模板（如颜色、尺寸、材质），通过预定义规格参数库实现商品

发布时快速调用与组合配置；商品管理提供完整的商品信息维护能力，新增商品需关联预设分类层级与分组标签，同步录入商品详情、价格、库存及规格参数，支持商品上下架状态切换、编辑，形成从标签创建、分类配置、规格定义到商品发布的完整业务流程，其业务结构图如 5.10 所示：

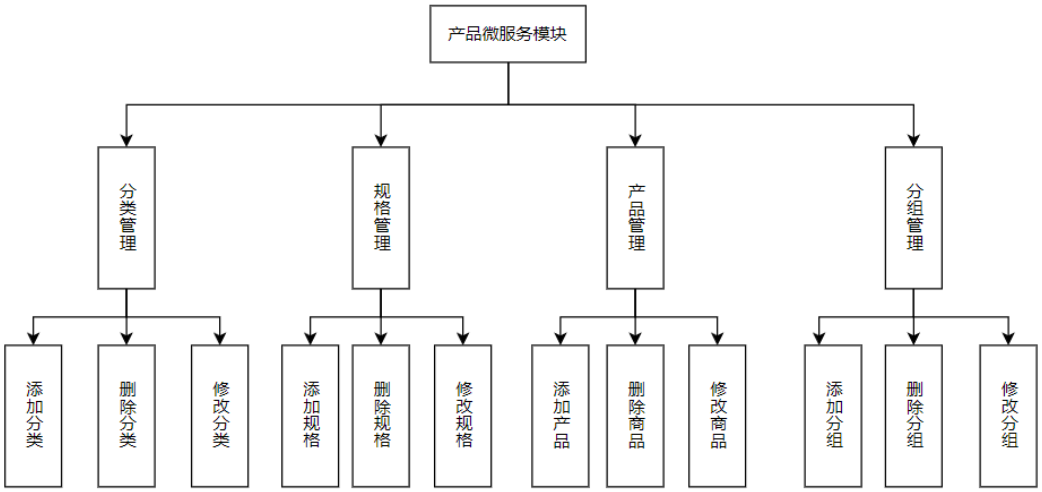

图 5.10 产品业务结构图

1.分组管理页面，列表显示所有分组标签信息，包括分组标签的名称、状态等，并且提供了查询标签、修改标签、删除标签的功能， 其中添加和修改可以设置分组标签下商品展示的样式。

2.分类管理页面下：列表显示所有的分类信息，包括分类名称、分类图片、分类状态等，且分类信息是按多级分类体系（如“服饰→女装”）结构化展示，管理员可后台维护分类树节点，支持添加子分类、调整层级顺序。

3.规格管理定义了商品通用属性模板（如颜色、尺寸、容量），规格管理页面下，列表显示所有的规格信息，包括属性 id、属性名称、属性值，管理员可以添加、删除、修改规格属性。

4.商品管理主要对商品信息进行管理和维护，管理页面列表显示所有商品信息，包括商品名字、价格、库存、商品图片等信息，同时包含新增、修改、删除商品功能。新增商品需绑定分类层级与分组标签，完整录入基础信息（标题、描述）、规格参数及库存、价格数据，且支持商品上下架功能。

5.2.9 门店微服务设计

门店服务核心功能主要负责区域化展示与动态内容管理，包括公告信息发布、首页轮播图显示、地址数据维护及用户热搜四大模块，支持通过公告管理实时编辑并同步首页顶部通知栏、配置轮播图素材与跳转链接实现可视化营销、维护全国省市区三级地址数据可以辅助用户精准定位，同时使用热搜关键词展示热搜详情，其业务结构图如 5.11 所示。

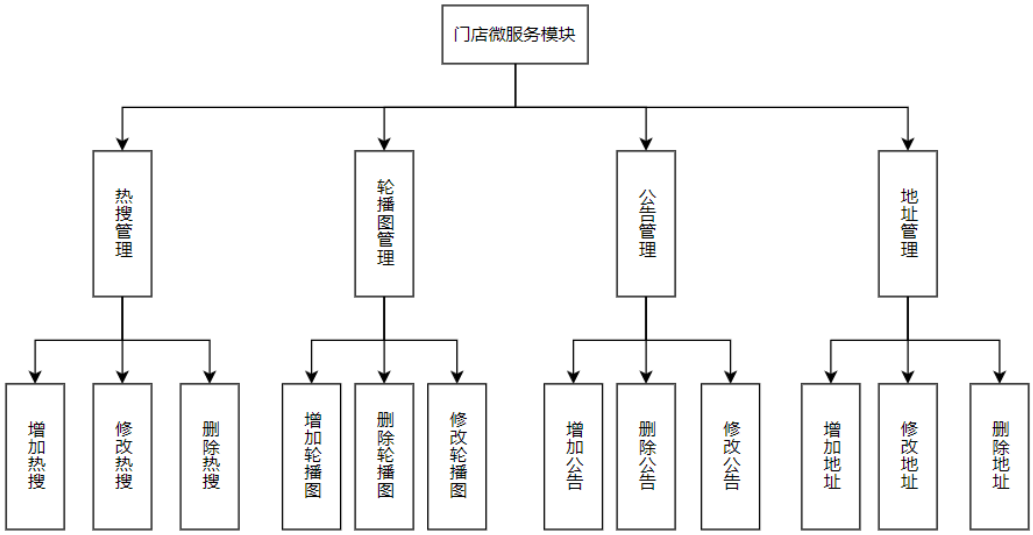

图 5.11 门店业务结构图

1.在公告管理页面，默认展示所有公告信息，并且支持按标题关键词检索公告条目、编辑公告内容以及删除公告功能，管理员可通过启停开关快速控制公告前端的可见性。

2.在轮播图管理页面，列表动态展示所有轮播图资源，支持上传/替换图片素材、配置点击跳转逻辑（商品详情页），以及删除轮播图功能。

3.在地址管理页面，以树形结构逐级展示全国省、市、区三级行政区划数据，允许管理员新增或调整行政区节点信息，确保用户下单时地址选择精准覆盖最新区域划分。管理员可以选择增加、删除修改地址信息。

4.在热搜管理页面，默认显示所有热搜信息，包括热搜标题、状态、内容等信息，管理员可以选择增加、修改、删除热搜。

6 商城购物系统实现

本章将基于需求分析与系统概要设计展开项目的详细设计与实现，并对核心功能模块进行深度解析。详细设计阶段的核心任务在于明确系统落地方案，通过微信商城平台与后台管理端的精细化设计，确保技术方案可直接转化为可执行的程序代码。本章将仔细阐明关键业务流程的执行逻辑，完整呈现商城平台各模块的设计细节与实现路径。

6.1 开发环境

商城购物系统开发中用到的软件环境如下所示：

表 6.1 系统开发软件工具与技术

类型	工具技术
微服务架构	SpringCloud Alibaba
开发工具	IntelliJ IDEA、Visual Studio Code、微信开发者工具
编程语言	Java、HTML、CSS、JavaScript、Vue
数据库存储	MySQL 8.0.32
缓存技术	Redis 7.0
文件存储技术	阿里云 OSS
版本控制工具	Git 2.48.1
Linux 环境	CentOS 7.0
消息通信	RabbitMQ 3.7.2

6.2 通用功能实现

通用功能主要包括微服务网关、服务注册和认证授权三个部分，下面将对对这三个部分进行一一描述。

1.微服务网关实现

微服务网关作为微信端与后台管理端流量的统一入口，通过动态路由与统一认证的深度整合，实现了安全校验与请求转发的无缝衔接。

当用户发起请求时，网关首先拦截用户请求，判断该请求路径是否在白名单内，如果在白名单内直接放行，如果请求路径不包含在白名单中，需要对其进行身份的验证，从约定好的位置获取 Authorization 的值，解析出 token 后，和 redis 中的 token 进行对比，如果一致便放行。

通过后基于服务发现机制动态匹配目标微服务名称，自动从注册中心获取可用实例后，并成负载均衡转发。

其核心代码如下：

```
// 获取请求对象
ServerHttpRequest request = exchange.getRequest();
// 获取请求路径
String path = request.getPath().toString();
// 判断当前请求路径是否需要放行，是否存在于白名单中
if (whiteUrlsConfig.getAllowUrls().contains(path)) {
// 包含：请求路径包含在白名单中，即需要放行
return chain.filter(exchange);
}
// 请求路径不包含在白名单中，需要对其进行身份的验证
// 从约定好的位置获取 Authorization 的值，值的格式为：bearer token
string  authorizationValue=
request.getHeaders().getFirst(AuthConstants.AUTHORIZATION)
// 判断是否有值
if (StringUtils.hasText(authorizationValue)) {
// 从 Authorization 的值中获取 token
String tokenValue = authorizationValue.replaceFirst(AuthConstants.BEARER, "");
// 判断 token 值是否有值且是否在 redis 中存在
if (StringUtils.hasText(tokenValue) &&
stringRedisTemplate.hasKey(AuthConstants.LOGIN_TOKEN_PREFIX +
tokenValue)) {
// 身份验证通过，放行
return chain.filter(exchange);
```

```
}
}
// 流程如果走到这：说明验证身份没有通过或请求不合法
log.error("拦截非法请求，时间:{},请求 API 路径:{}",new Date(),path);
```

2.服务注册

服务注册中心使用 Nacos,系统所有服务提供者包括网关本身都在此注册,由 Nacos 存储管理服务列表,并且对于一些公共配置，一并在 Nacos 中进行统一配置管理。服务提供者需要配置如下数据将服务和配置注册到 Nacos。

3.认证授权实现

用户在发起登陆时，经过网关放行后，由认证授权服务进行校验，因为该购物商城分为后台管理系统和微信小程序，需要设计两套不同的认证流程，可以根据请求头中携带的登录类型来判断登录来源。

如果是后台管理系统用户登录，前端将用户输入的用户名、密码信息传输到后台，后台用一个类对象将其封装起来，通常使用的是 UsernamePasswordAuthenticationToken 这个类。

程序负责验证这个类对象。验证方法是调用 Service，根据 username 从数据库中取用户信息到实体类的实例中，比较两者的密码，如果密码正确就成功登陆，同时把包含着用户的用户名、密码、所具有的权限等信息的类对象放到 SecurityContextHolder（安全上下文容器，类似 Session）中去。

用户访问一个资源的时候，首先判断是否是受限资源。如果是的话还要判断当前是否未登录，没有的话就跳到登录页面。

如果用户已经登录，访问一个受限资源的时候，程序要根据 url 去数据库中取出该资源所对应的所有可以访问的角色，然后拿着当前用户的所有角色一一对比，判断用户是否可以访问。

核心代码如下：

```
@Override
public UserDetails realLogin(String username) {
// 根据用户名称查询用户对象
LoginSysUser loginSysUser = loginSysUserMapper.selectOne(new
LambdaQueryWrapper<LoginSysUser>()
```

```
.eq(LoginSysUser::getUsername, username)
);
if (ObjectUtil.isNotNull(loginSysUser)) {
// 根据用户标识查询用户的权限集合
Set<String>perms =
loginSysUserMapper.selectPermsByUserId(loginSysUser.getUserId());
// 创建安全用户对象 SecurityUser
SecurityUser securityUser = new SecurityUser();
securityUser.setUserId(loginSysUser.getUserId());
securityUser.setPassword(loginSysUser.getPassword());
securityUser.setShopId(loginSysUser.getShopId());
securityUser.setStatus(loginSysUser.getStatus());
securityUser.setLoginType(AuthConstants.SYS_USER_LOGIN);
// 判断用户权限是否有值
if (CollectionUtil.isNotEmpty(perms) && perms.size() != 0) {
securityUser.setPerms(perms);
}
return securityUser;
}
}
```

登录成功后由登录成功处理器进行 token 的颁发，核心代码如下：

```
public AuthenticationSuccessHandler authenticationSuccessHandler() {
return (request, response, authentication) -> {
// 设置响应头信息
response.setContentType(HttpConstants.APPLICATION_JSON);
response.setCharacterEncoding(HttpConstants.UTF_8);
// 使用 UUID 来当作 TOKEN
String token = UUID.randomUUID().toString();
// 从 security 框架中获取认证用户对象(SecurityUser)并转换为 json 格式的字符串
```

```
String userJsonStr = JSONObject.toJSONString(authentication.getPrincipal());
// 将 token 当作 key，认证用户对象的 json 格式的字符串当前 value 存放到
redis 中
stringRedisTemplate.opsForValue().set(AuthConstants.LOGIN_TOKEN_PREFIX
+token,userJsonStr, Duration.ofSeconds(AuthConstants.TOKEN_TIME));
// 封装一个登录统一结果对象
LoginResult loginResult = new
LoginResult(token,AuthConstants.TOKEN_TIME);
// 创建一个响应结果对象 Result<Object> result = Result.success(loginResult);
// 返回结果
ObjectMapper objectMapper = new ObjectMapper();
String s = objectMapper.writeValueAsString(result);
PrintWriter writer = response.getWriter();
writer.write(s);
writer.flush();
writer.close();
};
}
```

如果是微信用户登录,流程和后台用户登录类似，微信小程序发送登陆请求，携带临时校验码和默认密码来换取 sessionKey 和 openId，如果根据 openId 去数据库里可以查询到用户数据，则登陆成功，然后由成功处理器颁发 token。

6.3 微信端核心业务实现

6.3.1 商品首页功能实现

微信用户登录成成功后，用户首先会进入首页，如图 6.1 所示。主要的功能分为搜索功能、轮播图功能、公告显示、商品展示与分组管理。

图 6.1 首页面图

1.搜索功能实现

用户点击搜索框，输入商品名称后向后台发起请求，根据输入内容进行模糊查询。其示意图如图 6.2 所示。

图 6.2 搜索页面图

其核心代码如下：

```
List<Prod> prods = null;
if(sort == 0){
QueryWrapper<Prod> queryWrapper = new QueryWrapper<>();
queryWrapper.like("prod_name",prodName);
queryWrapper.eq("status",1);
prods = prodMapper.selectList(queryWrapper);
}
```

```
if(sort == 1){
QueryWrapper<Prod> queryWrapper = new QueryWrapper<>();
queryWrapper.like("prod_name",prodName);
queryWrapper.eq("status",1);
queryWrapper.orderByDesc("sold_num");
prods = prodMapper.selectList(queryWrapper);
}
if(sort == 2){
QueryWrapper<Prod> queryWrapper = new QueryWrapper<>();
queryWrapper.like("prod_name",prodName);
queryWrapper.eq("status",1);
queryWrapper.orderByAsc("price");
prods = prodMapper.selectList(queryWrapper);
}
```

通过判断 sort 的值执行不同的条件查询，返回结果给前端。

2.轮播图

用户进入首页，会向后端发起请求获取轮播图信息以及所关联的商品的 id 值，用户点击轮播图则会跳转到对应的商品信息。

其核心代码如下：

```
// 获取轮播图
public List<IndexImg> queryWxIndexImgList() {
return indexImgMapper.selectList(new LambdaQueryWrapper<IndexImg>()
.eq(IndexImg::getStatus,1)
.orderByDesc(IndexImg::getSeq)
);
}
// 点击轮播跳转到商品详情页
// 根据标识查询商品详情
Prod prod = prodMapper.selectById(prodId);
if (ObjectUtil.isNull(prod)) {
```

```
return prod;
}
// 根据商品标识查询商品与分组标签的关系
List<ProdTagReference> prodTagReferenceList =
prodTagReferenceMapper.selectList(new
LambdaQueryWrapper<ProdTagReference>()
.eq(ProdTagReference::getProdId, prodId)
);
// 判断是否有值
if(CollectionUtil.isNotEmpty(prodTagReferenceList)&&
prodTagReferenceList.size() != 0) {
// 从商品与分组标签的关系集合中获取分组标签 id 集合
List<Long>tagIdList=prodTagReferenceList.stream().map(ProdTagReference::getT
agId).collect(Collectors.toList());
prod.setTagList(tagIdList);
}
// 根据商品 id 查询商品 sku 对象集合
List<Sku> skus = skuMapper.selectList(new LambdaQueryWrapper<Sku>()
.eq(Sku::getProdId, prodId)
);
prod.setSkuList(skus);
return prod;
```

3.分组显示

商品在首页按照分组标签展示，可以在后台管理系统调整每行显示的商品数量，如果用户需要了解更多该分组下的信息，可以点击右上角的更多按钮。更多商品页面如图 6.3 所示：

图 6.3 分组页面图

6.3.2 分类功能实现

商品分类页面集中展示全站商品的层级分类体系，页面默认呈现完整的分类树形结构（如服饰、家电、食品等大类及其子类），用户可以方便地浏览各个分类，快速定位感兴趣的商品。用户还可以通过点击分类名称，进一步筛选商品，查看该类下的所有相关商品，提升购物的效率。每个大类下的子类目将以可展开的树形方式呈现，方便用户查看具体商品。分类页面如图 6.4 所示。

图 6.4 分类页面图

核心代码如下：

```
// 查询小程序中商品的一级类目集合
@Override
@Cacheable(key = ProductConstants.WX_FIRST_CATEGORY)
public List<Category> queryWxCategoryListByPid(Long pid) {
return categoryMapper.selectList(new LambdaQueryWrapper<Category>()
.eq(Category::getStatus,1)
.eq(Category::getParentId,pid)
.orderByDesc(Category::getSeq)
);
}
// 根据商品类目标识查询商品集合
```

```
// 创建所有类目 id 集合
List<Long> allCategoryIds = new ArrayList<>();
allCategoryIds.add(categoryId);
// 远程调用：根据商品一级类目 id 查询子类目集合
Result<List<Category>>  ategoryResult =
searchProdFeign.getCategoryListByParentId(categoryId);
// 判断操作成功
if
(categoryResult.getCode().equals(BusinessEnum.OPERATION_FAIL.getCode()))
{
throw new BusinessException("远程调用失败：根据商品一级类目 id 查询子类
目集合");
}
// 获取数据
List<Category> categoryList = categoryResult.getData();
// 判断子类目集合是否有值
if (CollectionUtil.isNotEmpty(categoryList) && categoryList.size() != 0) {
// 从子类目集合中获取类目 id 集合
List<Long> categoryIdList =
categoryList.stream().map(Category::getCategoryId).collect(Collectors.toList());
allCategoryIds.addAll(categoryIdList);
}
// 远程调用：根据商品类目 id 集合查询商品对象集合
Result<List<Prod>> prodResult =
searchProdFeign.getProdListByCategoryIds(allCategoryIds);
// 判断查询结果
if (prodResult.getCode().equals(BusinessEnum.OPERATION_FAIL.getCode())) {
throw new BusinessException("远程调用失败：根据商品类目 id 集合查询商品
对象集合");
}
```

```
// 获取数据
List<Prod> prods = prodResult.getData();
return prods;
```

由于分类标签比较固定，变动不是很频繁，此处使用 redis 缓存，将分类名称存入 redis 中，下次用户再访问此页面不会向后端发起请求，而是直接从缓存中获取数据

6.3.3 商品详情功能

用户点击商品，将进入商品详情页，商品详情页会展示商品价格、简介、商品浏览图等，用户可以在此页面收藏商品、加入购物车、和立即购买详情页如图 6.5 所示：

图 6.5 商品详情页

1.商品详情页

用户进入商品详情页，向后端发起请求获取商品信息，用于回显商品信息，根据商品 id 查询商品详情，再根据商品 id 查询商品 sku 对象集合，把 sku 对象集合赋值给商品实体类，其核心代码如下：

```
// 根据标识查询商品详情
Prod prod = prodMapper.selectById(prodId);
if (ObjectUtil.isNull(prod)) {
return prod;
}
// 根据商品标识查询商品与分组标签的关系
List<ProdTagReference> prodTagReferenceList =
prodTagReferenceMapper.selectList(new
LambdaQueryWrapper<ProdTagReference>()
.eq(ProdTagReference::getProdId, prodId)
);
// 判断是否有值
if (CollectionUtil.isNotEmpty(prodTagReferenceList) &&
prodTagReferenceList.size() != 0) {
// 从商品与分组标签的关系集合中获取分组标签 id 集合
List<Long> tagIdList =
prodTagReferenceList.stream().map(ProdTagReference::getTagId).collect(Collector
s.toList());
prod.setTagList(tagIdList);
}
// 根据商品 id 查询商品 sku 对象集合
List<Sku> skus = skuMapper.selectList(new LambdaQueryWrapper<Sku>()
.eq(Sku::getProdId, prodId)
);
prod.setSkuList(skus);
```

其次，需要判断该商品是否收藏，根基商品 id 查询，其核心代码如下所示：

```
public  Result<Boolean> loadMemberIsCollection(@RequestParam Long prodId) {
long  count = memberCollectionService.count(new
LambdaQueryWrapper<MemberCollection>().eq(MemberCollection::getOpenId,
AuthUtils.getMemberOpenId())
```

```
.eq(MemberCollection::getProdId, prodId)
);
return  Result.success(1==count);
}
```

2.立即购买

用户选择立即购买，将跳转到订单详情页面，同时发起 ajax 请求，获取订单信息、地址信息返回给前台，地址栏显示默认设置地址，如果没有地址则无法提交订单。其订单详情页如图 6.6 所示：

图 6.6 购买页面图

核心代码如下：

```
// 创建订单确认页面对象
OrderVo orderVo = new OrderVo();
// 获取会员 openId
String openId = AuthUtils.getMemberOpenId();
```

```
// 远程调用：根据会员 openId 查询会员默认收货地址
Result<MemberAddr> result =
orderMemberFeign.getMemberDefaultAddrByOpenId(openId);
// 获取数据
MemberAddr memberAddr = result.getData();
orderVo.setMemberAddr(memberAddr);
// 从订单确认页面参数对象中获取购物车 id 集合
List<Long> basketIds = orderConfirmParam.getBasketIds();
// 判断订单确认页面请求来自于哪里？
if (CollectionUtils.isEmpty(basketIds) || basketIds.size() == 0) {
// 购物车 id 集合为空 -> 说明：请求来自于商品详情页面
productToOrderConfirm(orderConfirmParam.getOrderItem(), orderVo);
} else {
// 购物车 id 集合不为空 -> 说明：请求来自于购物车页面
orderVo.setSource(1);
cartToOrderConfirm(basketIds, orderVo);
}
```

3.提交订单

提交订单仅仅是把订单信息暂时存储到数据库中，用户需要前往订单栏中进行支付，在数据库中订单表里使用一个字段标识其支付状态，如果用户在规定时间内没有支付，将会取消订单，将订单状态标为未支付。

订单超时未支付取消订单的操作可以写法很多，可以使用订单任务，定期扫描订单表，查询到订单状态为未支付的订单则进行取消，这里采用 RabbitMQ 的 TTL 结合死信交换机来完成延时删除。

声明队列和交换机核心代码如下所示：

```
// 配置死信队列
@Bean
public Queue orderDeadQueue() {
return new Queue(QueueConstants.ORDER_DEAD_QUEUE);
}
```

```
// 配置交换机
@Bean
public DirectExchange orderDeadEx() {
return new DirectExchange(QueueConstants.ORDER_DEAD_EX);
}
// 配置延迟队列
@Bean
public Queue orderMsQueue() {
// 配置延迟队列参数
Map<String,Object> map = new HashMap<>();
// 配置消息存活时长，15 分钟
map.put("x-message-ttl", 60*1000*15);
// 配置消息死后走的交换机
map.put("x-dead-letter-exchange", QueueConstants.ORDER_DEAD_EX);
// 消息死后走的路由 key
map.put("x-dead-letter-routing-key",QueueConstants.ORDER_DEAD_KEY);
return new Queue(QueueConstants.ORDER_MS_QUEUE,true, false,false,map);
}
@Bean
public Binding orderDeadBind() {
return
BindingBuilder.bind(orderDeadQueue()).to(orderDeadEx()).with(QueueConstants.
ORDER_DEAD_KEY);
}
```

4.加入购物车

用户可以将商品商品放入购物车中，需要传递用户 id、产品 id、商品属性 id、购物车产品数量到后端，根据会员 openId 和商品 skuId 查询购物车记录，如果存在记录的话说明购物车中存在该商品，直接修改存在购物车中商品的数量，如果购物车记录为空，表示当前用户添加到购物车中的商品是不存在的，直接添

加商品到购物车记录。

其核心代码如下：

```
// 获取会员 openid
String openId = AuthUtils.getMemberOpenId();
// 根据会员 openId 和商品 skuId 查询购物车记录
Basket beforeBasket = basketMapper.selectOne(new
LambdaQueryWrapper<Basket>()
.eq(Basket::getOpenId, openId)
.eq(Basket::getSkuId, basket.getSkuId())
);
// 判断购物车记录是否有值
if (ObjectUtil.isNotNull(beforeBasket)) {
// 购物车记录不为空 -> 当前会员添加到购物车中的商品是存在的 -> 修改
存在购物车中商品的数量
// 计算商品最终数量
int finalCount = beforeBasket.getProdCount() + basket.getProdCount();
beforeBasket.setProdCount(finalCount);
return basketMapper.updateById(beforeBasket)>0;
}
// 购物车记录为空 -> 当前会员添加到购物车中的商品是不存在的 -> 添加
商品到购物车记录
basket.setCreateTime(new Date());
basket.setOpenId(openId);
return basketMapper.insert(basket)>0;
```

5.收藏

根据商品 id 去数据库收藏表中查询，数据不为空说明已经收藏，则把状态改为未收藏，数据为空说明未收藏，把状态改为收藏，其核心代码如下所示：

```
MemberCollection memberCollection = memberCollectionMapper.selectOne(new
LambdaQueryWrapper<MemberCollection>()
.eq(MemberCollection::getOpenId, openId)
```

```
.eq(MemberCollection::getProdId, prodId)
);
if (ObjectUtil.isNull(memberCollection)) {
memberCollection = new MemberCollection();
memberCollection.setCreateTime(new Date());
memberCollection.setOpenId(openId);
memberCollection.setProdId(prodId);
return memberCollectionMapper.insert(memberCollection)>0;
}
return memberCollectionMapper.deleteById(memberCollection.getId())>0;
```

6.3.4 购物车功能实现

用户添加到购物车中的商品将在购物车页面展示，其购物车页面如图 6.7 所示：

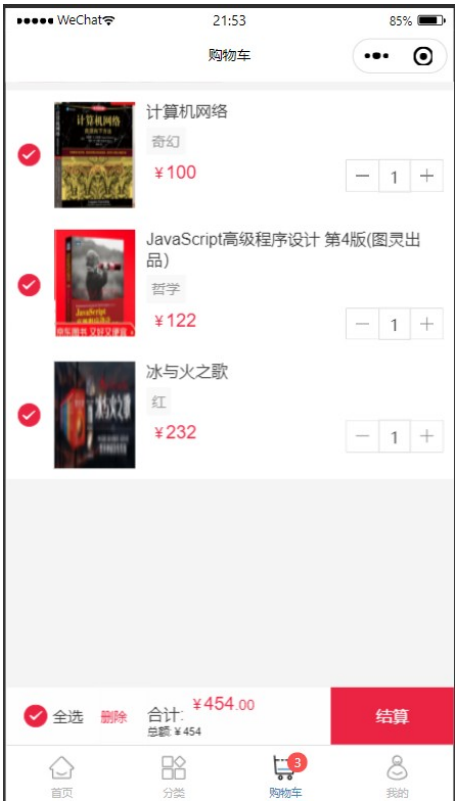

图 6.7 购物车页面图

1.修改商品数量

修改商品数量和添加商品到购物车的实现过程一样，因为根据用户 id 和商品 id 去数据库中可以查询到数据，所以直接修改商品在购物车中的数量。

2.商品购物车

根据购物车记录 id 去数据库中删除，可能会删除删除多个购物车记录，后端需要使用集合接收。

3.统计金额

根据所选购物车记录 id 查询购物车对象集合，循环遍历购物车对象集合，获取购物车记录中商品购买的数量，获取商品单价，计算单个商品总金额，添加到单个商品总金额集合中，计算所有单个商品总金额的和。

其核心代码如下:

```
// 创建购物车总金额对象
CartTotalAmount cartTotalAmount = new CartTotalAmount();
// 判断会员是否有选中的购物车记录
if (CollectionUtils.isEmpty(basketIds) || basketIds.size() == 0) {
// 购物车 id 集合为空 -> 说明会员没有选中购物车记录 -> 购物车商品金额为 0
return cartTotalAmount;
}
// 购物车 id 集合不为空 -> 说明会员有选中的购物车记录 -> 计算金额
// 根据购物车 id 集合查询购物车对象集合
List<Basket> basketList = basketMapper.selectBatchIds(basketIds);
// 从购物车对象集合中获取商品 skuId 集合
List<Long> skuIdList =
basketList.stream().map(Basket::getSkuId).collect(Collectors.toList());
// 远程调用：根据商品 skuId 集合查询商品 sku 对象集合
Result<List<Sku>> result = basketProdFeign.getSkuListBySkuIds(skuIdList);
// 判断查询结果
```

```
if (result.getCode().equals(BusinessEnum.OPERATION_FAIL.getCode())) {
throw new BusinessException("远程调用：根据商品 skuId 集合查询商品 sku 对象集合失败");
}
// 获取返回数据
List<Sku> skuList = result.getData();
// 创建所有单个商品总金额的集合
List<BigDecimal> oneSkuTotalAmounts = new ArrayList<>();
// 从商品 sku 对象集合中获取与当前购物车记录的 skuId 一致的购物车记录对象
Sku sku1 = skuList.stream()
.filter(sku -> sku.getSkuId().equals(skuId))
.collect(Collectors.toList()).get(0);
// 获取商品单价
BigDecimal price = sku1.getPrice();
// 计算单个商品总金额
BigDecimal oneSkuTotalAmount = price.multiply(new BigDecimal(prodCount));
// 添加到单个商品总金额集合中
oneSkuTotalAmounts.add(oneSkuTotalAmount);
});
// 计算所有单个商品总金额的和
BigDecimal allSkuTotalAmount =
oneSkuTotalAmounts.stream().reduce(BigDecimal::add).get();
// 填充数据
cartTotalAmount.setTotalMoney(allSkuTotalAmount);
cartTotalAmount.setFinalMoney(allSkuTotalAmount);
if (allSkuTotalAmount.compareTo(new BigDecimal(99)) == -1) {
cartTotalAmount.setTransMoney(new BigDecimal(6));
cartTotalAmount.setFinalMoney(allSkuTotalAmount.add(new BigDecimal(6)));
}
```

return cartTotalAmount;

4.结算功能和提交订单功能类似，会将选中的购物车商品集合打包成 1 个订单进行提交。提交的订单详情如图 6.8 所示：

图 6.8 结算多种商品页面图

6.3.5 个人中心功能实现

个人中心功能包含收藏、查看订单、地址管理三个部分，涉及到的功能包括查看未支付订单、查看已完成订单、查看收藏、地址管理等，其页面如图 6.9 所示：

图 6.9 个人中心页面图

1.我的订单

我的订单页面会展示待支付、已取消、已完成的订单，对于待支付的订单，可以选择取消或者支付。其示意图如 6.10 所示：

图 6.10 订单页面图

取消订单采取的是逻辑删除，后端根据提供的订单编号，将数据库中对应的订单状态改为支付失败

其核心代码如下所示：

```
QueryWrapper<Order> objectQueryWrapper = new QueryWrapper<>();
objectQueryWrapper.eq("order_number", orderNumber);
Order order = new Order();
order.setUpdateTime(new Date());
order.setStatus(6);
int update = orderMapper.update(order, objectQueryWrapper);
if (update   == 1){
return true;
}
return false;
```

订单支付则是把支付状态改为已支付，其核心代码如下所示：

```
public Boolean toPay(PayMethod payMethod) {
QueryWrapper<Order> wrapper = new QueryWrapper<>();
wrapper.eq("order_number", payMethod.getOrderNumber());
Order order = new Order();
order.setPayTime(new Date());
order.setIsPayed(1);
order.setStatus(5);
int update = orderMapper.update(order, wrapper);
return update !=0;
}
```

2.收藏页

查询收藏表中所有数据，交给前端渲染，其示意图所图 6.11 所示：

图 6.11 收藏页面图

3.地址管理

其涉及到五个接口，展示地址、添加地址、修改地址、删除地址、设为默认地址。其所需要的核心数据如实体类所示：

```
// ID
private Long addrId;
// 收货人
private String receiver;
// 省 ID
private Long provinceId;
// 省
private String province;
// 城市
private String city;
// 城市 ID
private Long cityId;
// 区
private String area;
// 区 ID
private Long areaId;
// 邮编
private String postCode;
// 地址
private String addr;
// 手机
private String mobile;
// 是否默认地址 1 是
private Integer commonAddr;
```

图 6.12 和图 6.13 分别是地址展示页面和修改删除页面。

图 6.12 地址展示页面图

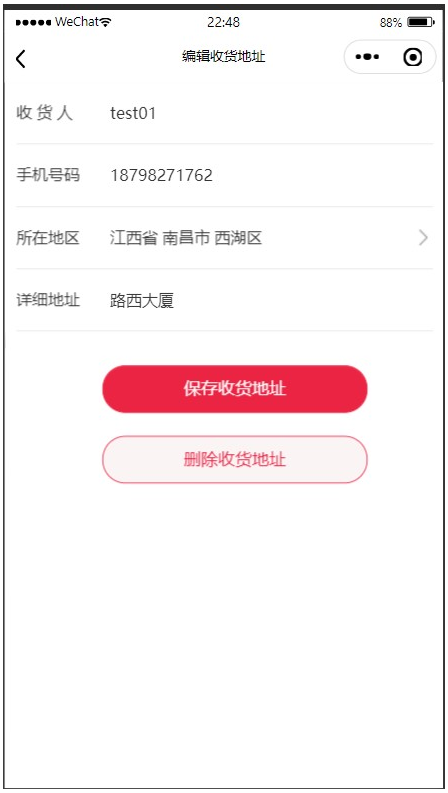

图 6.13 修改删除页面图

6.4 后台管理端核心业务实现

6.4.1 平台登录页面

当用户使用浏览器访问 http://localhost:8001 时，用户将进入登陆界面进行登录。用户输入正确的用户名和密码后，将会被颁发 token,后续的请求都会携带该 token。图 6.14 和图 6.15 分别表示登录页面和 token 信息

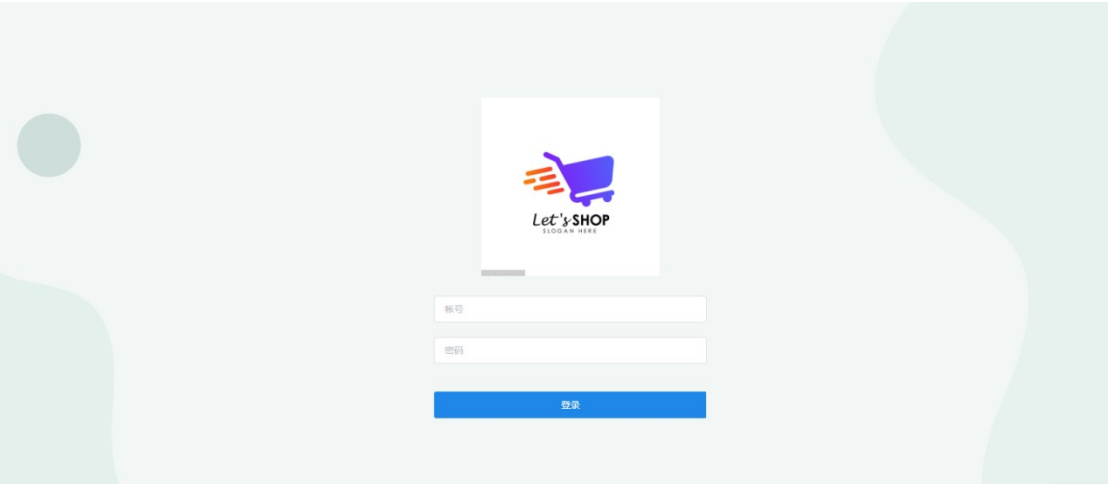

图 6.14 登录页面图

密钥	值
Authorization	bearer a0f4cb88-ece8-43f2-b13c-1bece79cfee1
loglevel:webpack-dev-server	WARN

图 6.15 token 信息页面图

核心代码如下：

```
// 根据用户名称查询用户对象
LoginSysUser loginSysUser = loginSysUserMapper.selectOne(new
LambdaQueryWrapper<LoginSysUser>()
.eq(LoginSysUser::getUsername, username)
);
if (ObjectUtil.isNotNull(loginSysUser)) {
```

```
// 根据用户标识查询用户的权限集合
Set<String> perms =
loginSysUserMapper.selectPermsByUserId(loginSysUser.getUserId());
// 创建安全用户对象 SecurityUser
SecurityUser securityUser = new SecurityUser();
securityUser.setUserId(loginSysUser.getUserId());
securityUser.setPassword(loginSysUser.getPassword());
securityUser.setShopId(loginSysUser.getShopId());
securityUser.setStatus(loginSysUser.getStatus());
securityUser.setLoginType(AuthConstants.SYS_USER_LOGIN);
// 判断用户权限是否有值
if (CollectionUtil.isNotEmpty(perms) && perms.size() != 0) {
securityUser.setPerms(perms);
}
```

6.4.2 用户管理模块实现

进入用户管理页面，将显示全部的用户信息，用户还可以通过姓名、状态来精确查询某一个员工信息。用户也可以通过修改用户状态禁用或启动用户，同时也可以通过删除移除用户。员工管理页面如下图 6.16 所示：

图 6.16 员工管理页面图

其核心代码如下：

```
public Result<Page<Member>> loadMemberPage(@RequestParam Long current,
@RequestParam Long size,
@RequestParam(required = false) String nickName,
```

```
@RequestParam(required = false) Integer status) {
// 创建会员分页对象
Page<Member> page = new Page<>(current,size);
// 多条件分页查询会员
page = memberService.page(page,new LambdaQueryWrapper<Member>()
.eq(ObjectUtil.isNotNull(status),Member::getStatus,status)
.like(StringUtils.hasText(nickName),Member::getNickName,nickName)
.orderByDesc(Member::getCreateTime)
);
return Result.success(page);
```

先通过 ajax 发送 http 请求给后端系统，携带分页数据，以及用户名和状态，后端系统查询所有的用户数据封装成为一个 list 返回给前台页面，前台在经过自己的加工处理后显示。

6.4.3 订单管理模块实现

订单管理页面用户主要的功能可以通过订单编号、订单日期、订单状态查询某个订单，用户可以进入订单详情页，可以查看用户订购信息以及地址信息，且用户可以导出完成状态的订单。

图 6.17 和图 6.18 分别是订单管理页面和订单详情表单。

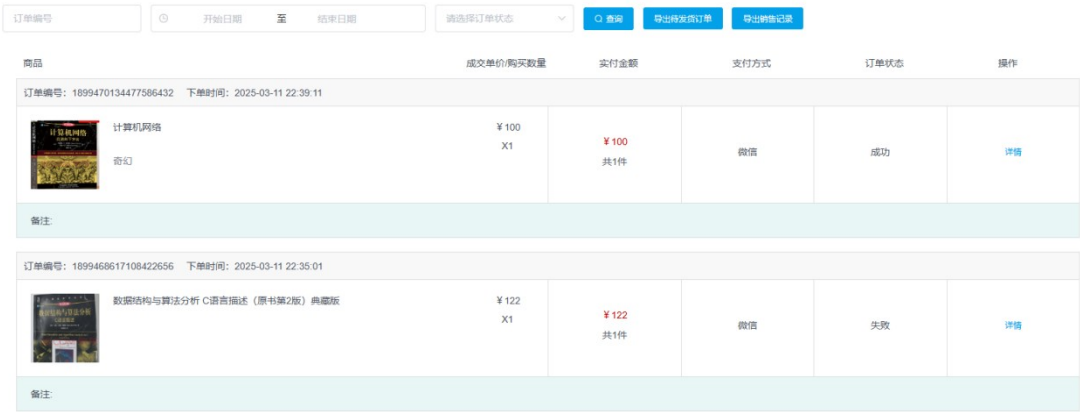

图 6.17 订单管理页面图

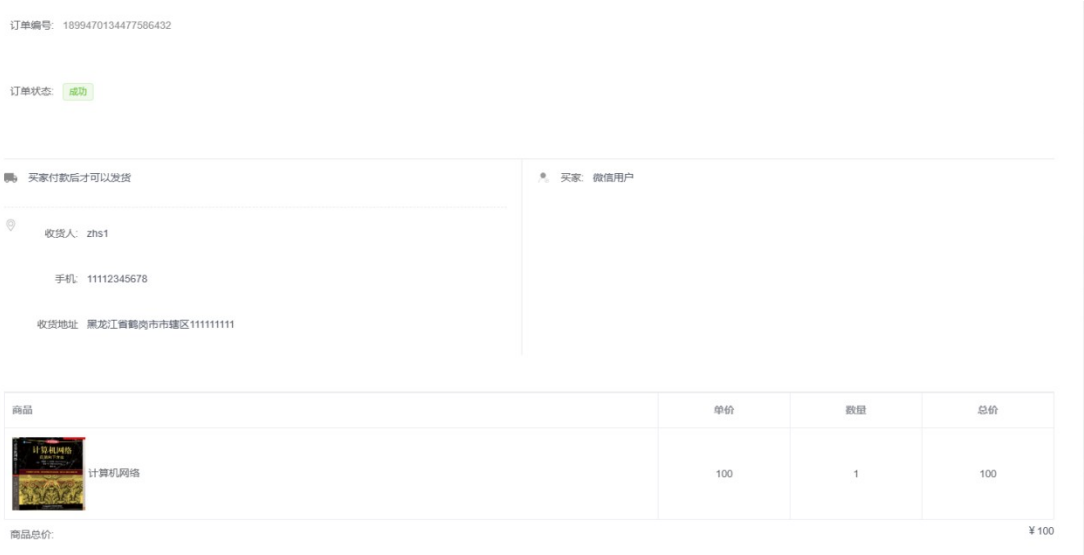

图 6.18 订单详情图

核心代码如下：

```
// 多条件分页查询订单列表
page = orderMapper.selectPage(page, new LambdaQueryWrapper<Order>()
.eq(ObjectUtil.isNotNull(status), Order::getStatus, status)
.between(ObjectUtil.isAllNotEmpty(startTime, endTime), Order::getCreateTime,
startTime, endTime)
.eq(StringUtils.hasText(orderNumber), Order::getOrderNumber, orderNumber)
.orderByDesc(Order::getCreateTime)
);
// 从订单分页对象中获取订单记录
List<Order> orderList = page.getRecords();
// 判断是否有值
if (CollectionUtils.isEmpty(orderList) || orderList.size() == 0) {
return page;
}
// 从订单记录集合中获取订单编号集合
List<String> orderNumberList =
orderList.stream().map(Order::getOrderNumber).collect(Collectors.toList());
// 根据订单编号查询所有订单商品条目对象集合
```

```
List<OrderItem> orderItemList = orderItemMapper.selectList(new
LambdaQueryWrapper<OrderItem>()
.in(OrderItem::getOrderNumber, orderNumberList)
);
// 循环遍历订单记录集合
orderList.forEach(order -> {
// 从订单商品条目对象集合中过滤出与当前订单记录的编号一致的商品条目对象集合
List<OrderItem> itemList = orderItemList.stream()
.filter(orderItem -> orderItem.getOrderNumber().equals(order.getOrderNumber()))
.collect(Collectors.toList());
});
```

6.4.4 系统管理模块设计

系统管理模块为商城运营提供管控能力，主要包含以下四个功能单元，即系统日志、菜单、管理员和角色。具体页面和实现如下所示。

1.系统日志

系统日志页面展示系统日志记录，包括操作用户 id、用户操作、请求方法和执行时长等，且可以通过用户 id 和用户操作查询某条日志。

图 6.19 表示日志管理页面

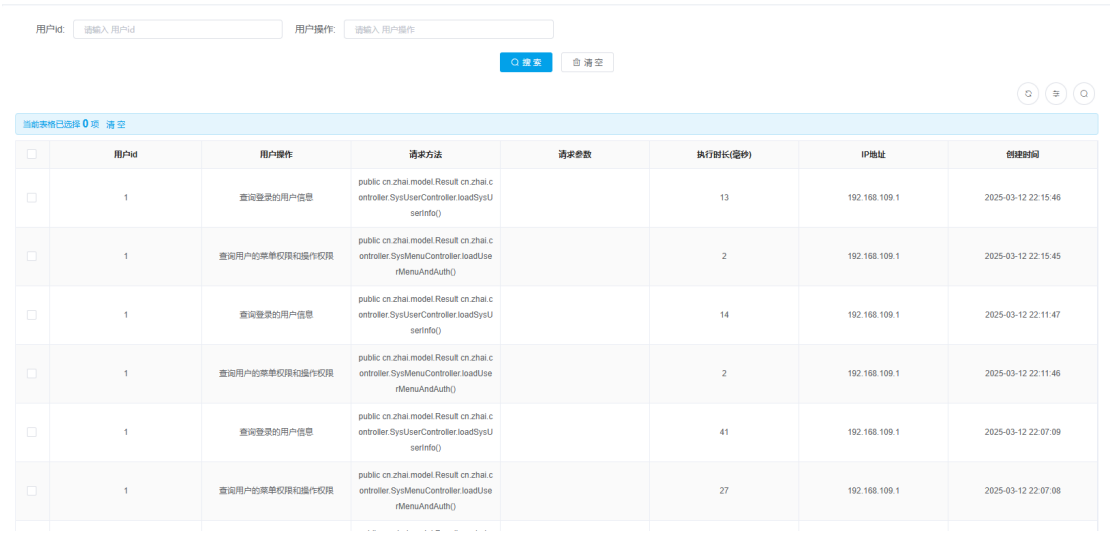

图 6.19 日志管理页面图

核心代码如下：

```
@Component
@Aspect
@Slf4j
public class SysLogAspect {
@Autowired
private SysLogService sysLogService;
// 切入点表达式
public static final String POINT_CUT = "execution (* cn.zhai.controller.*.*(..))";
@Around(value = POINT_CUT)
public Object logAround(ProceedingJoinPoint joinPoint) {
Object result = null;
// 获取请求对象
ServletRequestAttributes requestAttributes = (ServletRequestAttributes)
RequestContextHolder.getRequestAttributes();
HttpServletRequest request = requestAttributes.getRequest();
// 获取请求路径
String path = request.getRequestURI();
// 获取 ip 地址
String remoteHost = request.getRemoteHost();
// 获取请求方法
MethodSignature signature = (MethodSignature) joinPoint.getSignature();
Method method = signature.getMethod();
String methodName = method.toString();
// 获取目标方法上的 ApiOperation 注解
ApiOperation apiOperation =
method.getDeclaredAnnotation(ApiOperation.class);
// 获取 apiOperation 注解的描述
operation = apiOperation.value();
```

```
}
String finalArgs = "";
// 判断参数类型
if (ObjectUtil.isNotNull(args) && args.length != 0 && args[0] instanceof
MultipartFile) {
// 说明当前参数为文件对象
finalArgs = "file";
} else if (ObjectUtil.isEmpty(args)) {
finalArgs = "";
} else {
// 将参数对象转换为 json 格式的字符串
finalArgs = JSONObject.toJSONString(args);
}
// 记录开始时间
long startTime = System.currentTimeMillis();
try {
// 执行方法
result = joinPoint.proceed(args);
    } catch (Throwable e) {
    throw new RuntimeException(e);
    }
    // 记录结束时间
    long endTime = System.currentTimeMillis();
    // 方法执行时长
    long execTime = endTime - startTime;
    // 打印日志
    log.info("调用时间:{},请求接口路径:{},请求 IP 地址:{},方法名称:{},执行
    时长:{},方法描述:{}",
    new Date(),
    path,
```

```
remoteHost,
methodName,
execTime,
operation);
}
}
```

通过自定义切面类实现日志记录，通过切入点搭理控制器方法，在方法执行前计时、获取请求参数等，最后把数据存入数据库。

2.菜单管理

用户可以通过菜单名称模糊查询、添加新的菜单项、删除已有菜单、修改现有菜单配置。系统支持动态配置菜单权限，确保不同角色有不同的菜单访问权限。图 6.20 展示了菜单管理页面

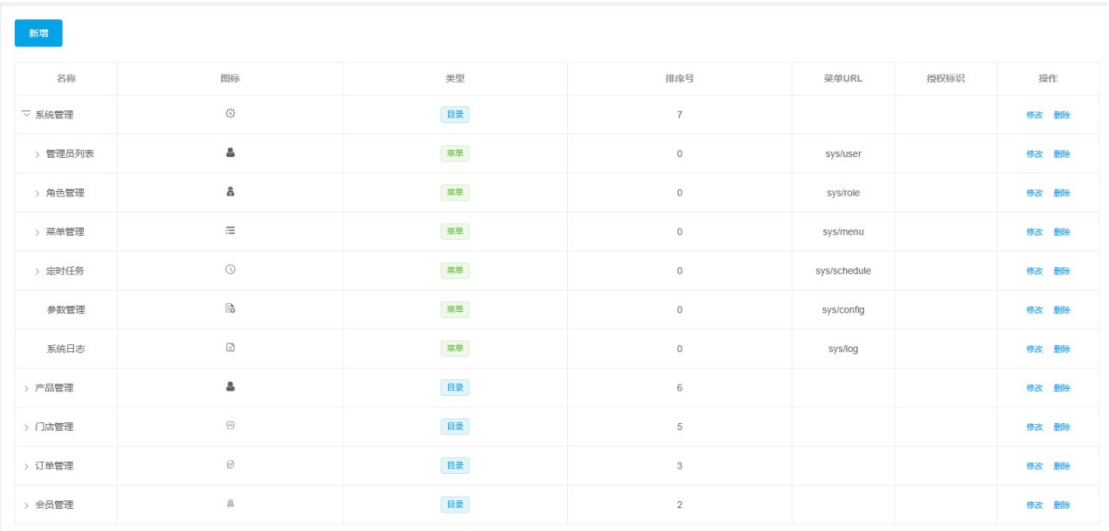

图 6.20 菜单管理页面图

其核心代码如下：

```
// 获取当前登录用户的标识
Long loginUserId = AuthUtils.getLoginUserId();
// 根据用户标识查询操作权限集合
Set<String> perms = AuthUtils.getLoginUserPerms();
// 根据用户标识查询菜单权限集合
Set<SysMenu> menus =
```

```
sysMenuService.queryUserMenuListByUserId(loginUserId);
// 创建菜单和操作权限对象
MenuAndAuth menuAndAuth = new MenuAndAuth(menus,perms);
return Result.success(menuAndAuth);
```

3.管理员管理

用户可以查看管理员信息，并对管理员进行新增、删除、修改操作。管理员的角色和权限可以根据需求进行调整。图 6.21 和图 6.22 分别显示了管理员管理页面以及管理员新增表单。

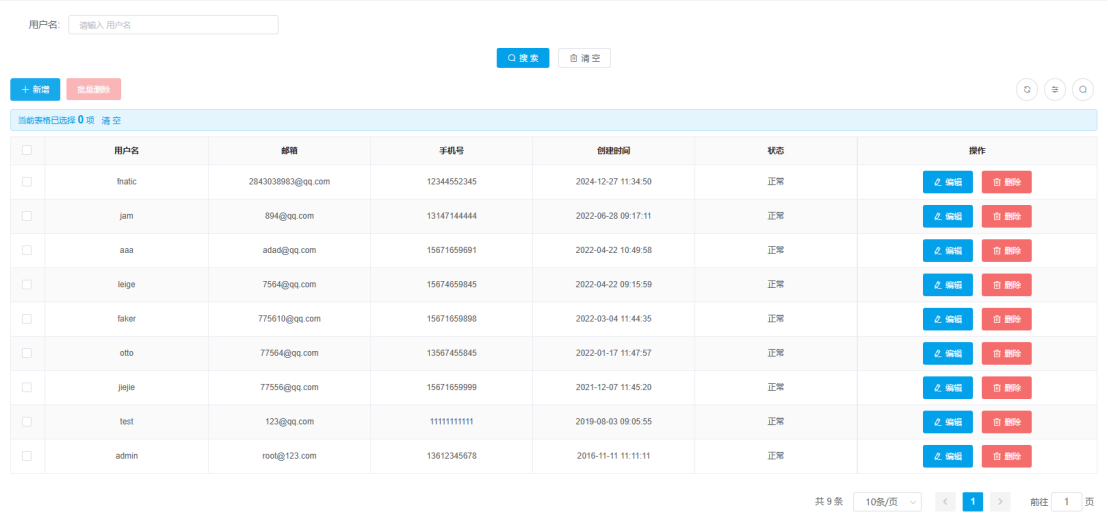

图 6.21 管理员管理页面

图 6.22 管理员新增表单

分页查询管理员列表核心代码如下所示：

```
// 创建 Mybatisplus 的分页对象
Page<SysUser> page = new Page<>(current, size);
// 多条件分页查询系统管理员
page = sysUserService.page(page, new LambdaQueryWrapper<SysUser>()
.like(StringUtils.hasText(username), SysUser::getUsername, username)
.orderByDesc(SysUser::getCreateTime)
);
return Result.success(page);
```

新增管理员列表核心代码如下：

```
// 新增管理员
sysUser.setCreateUserId(AuthUtils.getLoginUserId());
sysUser.setCreateTime(new Date());
sysUser.setShopId(1L);
sysUser.setPassword(passwordEncoder.encode(sysUser.getPassword()));
int i = sysUserMapper.insert(sysUser);
if (i > 0) {
// 获取管理员标识
Long userId = sysUser.getUserId();
// 新增管理员与角色的关系
// 获取管理员的角色 id 集合
List<Long> roleIdList = sysUser.getRoleIdList();
System.err.println(sysUser);
// 判断是否有值
if (CollectionUtil.isNotEmpty(roleIdList) && roleIdList.size() != 0) {
// 创建管理员与角色关系的集合
List<SysUserRole> sysUserRoleList = new ArrayList<>();
// 循环遍历角色 id 集合
roleIdList.forEach(roleId -> {
```

```
// 创建管理员与角色的关系
SysUserRole sysUserRole = new SysUserRole();
sysUserRole.setUserId(userId);
sysUserRole.setRoleId(roleId);
// 新增管理员与角色的关系
sysUserRoleMapper.insert(sysUserRole);*/
sysUserRoleList.add(sysUserRole);
});
// 批量添加管理员与角色的关系
sysUserRoleService.saveBatch(sysUserRoleList);
}
}
return i;
```

4.角色管理

在角色管理页面，默认展示所有的角色信息外，并且提供了创建、修改和删除角色的按钮。新增会修改中最重要的是设置该角色拥有的权限范围，每个角色都对应一组特定的权限。其分页展示和新增页面如图 6.23 和图 6.24 所示：

图 6.23 分页展示页面

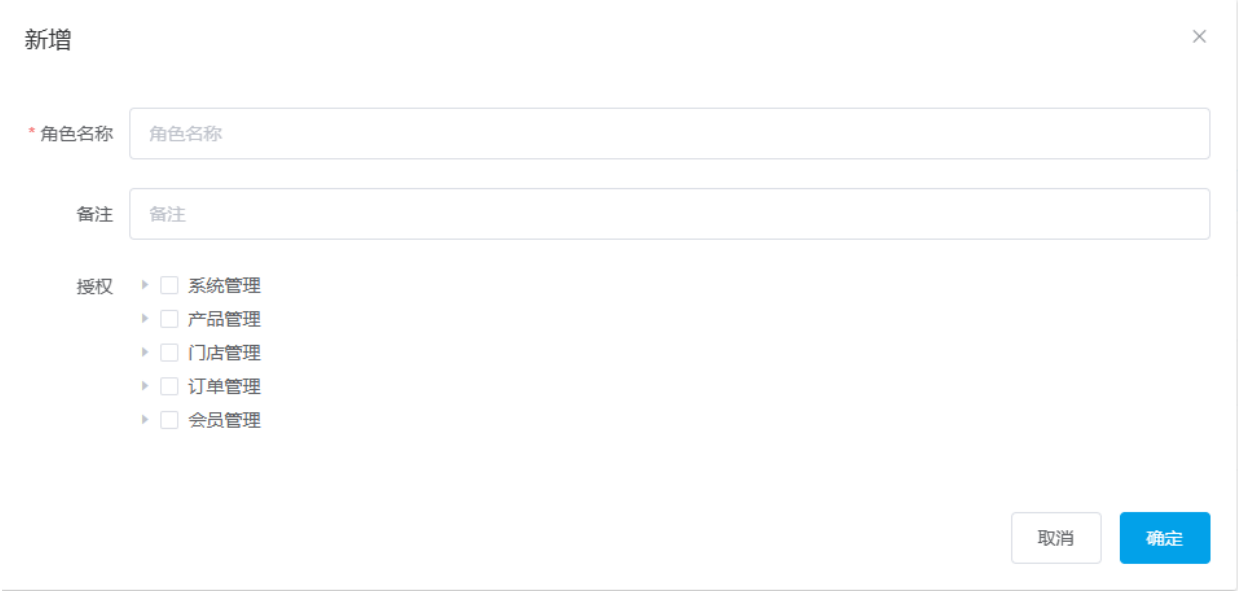

图 6.24 新增页面图

角色管理中值重要的就是对每个角色进行授权，可以给不同的角色设置不同的权限，其核心代码如下：

```
// 新增角色
sysRole.setCreateTime(new Date());
sysRole.setCreateUserId(AuthUtils.getLoginUserId());
int i = sysRoleMapper.insert(sysRole);
if (i > 0) {
// 获取角色 id
Long roleId = sysRole.getRoleId();
// 新增角色与权限的关系记录
// 获取角色对应的权限 id 集合
List<Long> menuIdList = sysRole.getMenuIdList();
// 创建角色与权限关系集合对象
List<SysRoleMenu> sysRoleMenuList = new ArrayList<>();
// 判断是否有值
if (CollectionUtil.isNotEmpty(menuIdList) && menuIdList.size() != 0) {
// 循环遍历权限 id 集合
menuIdList.forEach(menuId -> {
```

```
// 创建角色与权限关系记录
SysRoleMenu sysRoleMenu = new SysRoleMenu();
sysRoleMenu.setRoleId(roleId);
sysRoleMenu.setMenuId(menuId);
// 收集角色与权限的关系记录
sysRoleMenuList.add(sysRoleMenu);
});
// 批量添加角色与权限关系集合
sysRoleMenuService.saveBatch(sysRoleMenuList);
}
}
return i>0;
```

在新增中，前端携带了 menuIdList 这个权限集合，除了保存角色信息到数据库外，还需要保存角色和权限的对应关系，因此需要一个中间表，其代码如下：

```
public class SysRoleMenu implements Serializable {
@TableId(value = "id", type = IdType.AUTO)
@ApiModelProperty(value="")
private Long id;
// 角色 ID
@TableField(value = "role_id")
@ApiModelProperty(value="角色 ID")
private Long roleId;
// 菜单 ID
@TableField(value = "menu_id")
@ApiModelProperty(value="菜单 ID")
private Long menuId;
}
```

6.4.5 产品管理模块设计

进入产品管理模块，主要展示以下四个功能单元，即规格管理、产品管理、分类管理和分组管理。具体页面和实现如下所示。

1.规格管理

用户进入规格管理，将展示所有规格信息，包括属性 ID、属性名称、属性值，可以通过属性名称模糊查询某个属性，还可以新增属性、删除属性、修改属性。其规格页面如图 6.25 所示：

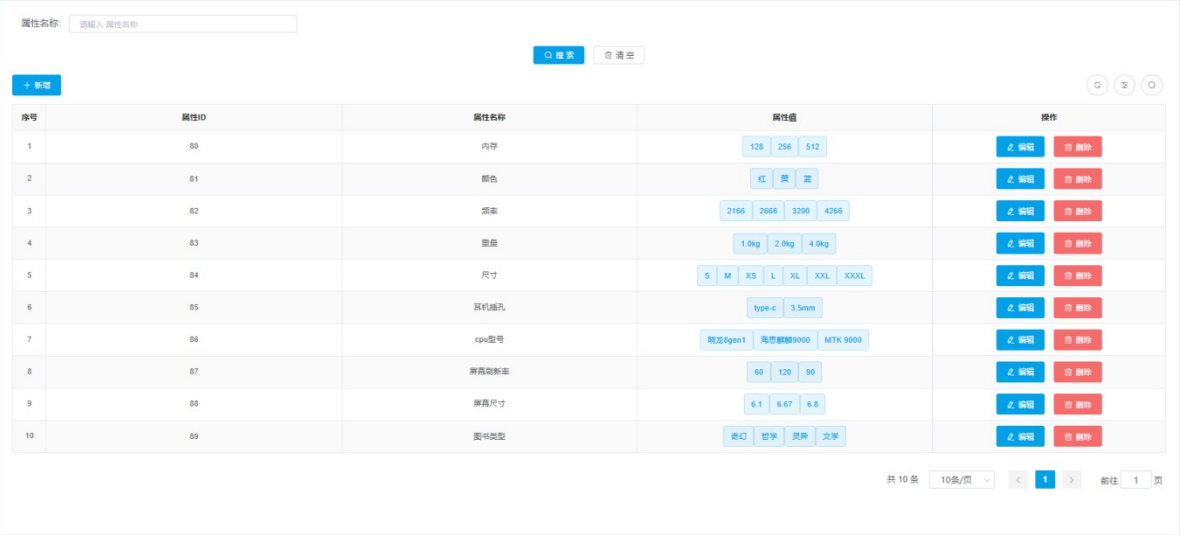

图 6.25 规格页面图

核心代码实现：

```
// 创建分页对象
Page<ProdProp> page = new Page<>(current,size);
// 多条件分页查询商品属性
page = prodPropMapper.selectPage(page,new
LambdaQueryWrapper<ProdProp>().like(StringUtils.hasText(propName),ProdProp
::getPropName,propName)
);
// 从分页对象中获取属性记录
List<ProdProp> prodPropList = page.getRecords();
// 判断是否有值
if (CollectionUtil.isEmpty(prodPropList) || prodPropList.size() == 0) {
```

```
// 如果属性对象集合没有值，说明属性值也为空
return page;
}
// 从属性对象集合中获取属性 id 集合
List<Long> propIdList =
prodPropList.stream().map(ProdProp::getPropId).collect(Collectors.toList());
// 属性 id 集合查询属性值对象集合
List<ProdPropValue> prodPropValueList = prodPropValueMapper.selectList(new
LambdaQueryWrapper<ProdPropValue>()
.in(ProdPropValue::getPropId, propIdList)
);
// 循环遍历属性对象集合
prodPropList.forEach(prodProp -> {
// 从属性值对象集合中过滤出与当前属性对象的属性 id 一致的属性对象集合
List<ProdPropValue> propValues = prodPropValueList.stream()
.filter(prodPropValue ->
prodPropValue.getPropId().equals(prodProp.getPropId())).collect(Collectors.toList(
));
prodProp.setProdPropValues(propValues);
});
return page;
```

2.分组管理

分组管理页面主要的功能可以通过标签名字模糊查询某个分组，支持新增分组、删除分组、修改分组。图 6.26 分组管理页面。

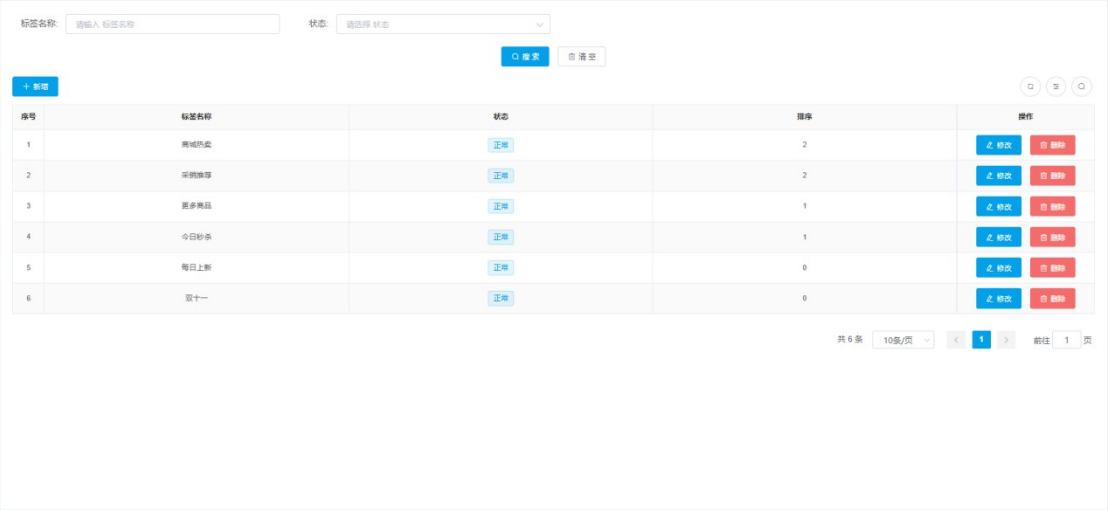

图 6.26 分组管理页面图

3.分类管理

进入分页管理，通过树形图的方式展示商品分类，用户可以选择新增、删除、修改节点，其分类示意图和新增示意图所图 6.27 和图 6.28 所示：

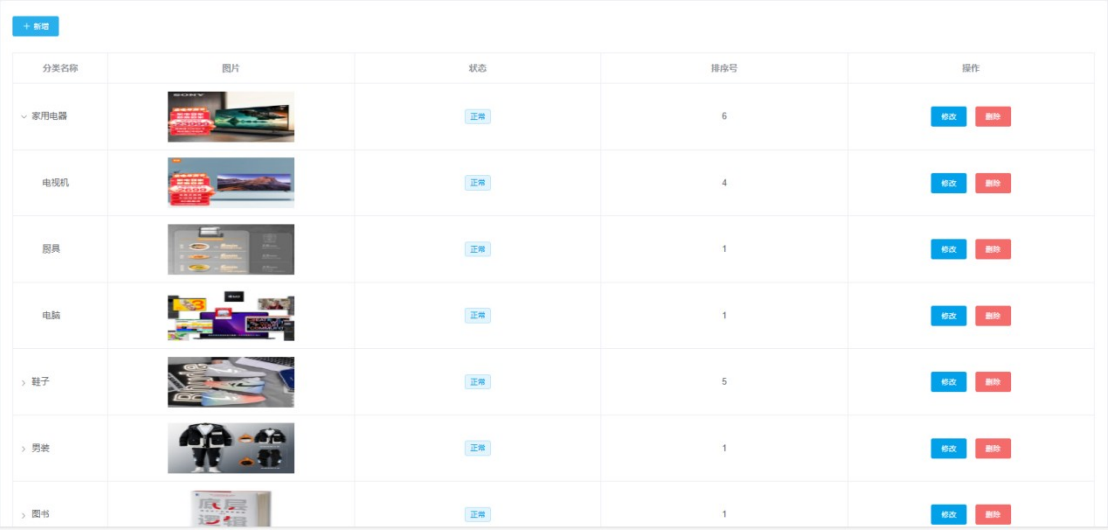

图 6.27 分类页面图

图 6.28 新增页面图

核心代码如下：

```
@Caching(evict = {
@CacheEvict(key = ProductConstants.ALL_CATEGORY_LIST_KEY),
@CacheEvict(key = ProductConstants.FIRST_CATEGORY_LIST_KEY),
@CacheEvict(key = ProductConstants.WX_FIRST_CATEGORY)
})
public Boolean saveCategory(Category category) {
category.setCreateTime(new Date());
category.setUpdateTime(new Date());
return categoryMapper.insert(category)>0;
}
```

由于分类标签变动不是很频繁，此处使用 redis 缓存，将分类名称存入 redis 中，下次用户再访问此页面不会向后端发起请求，而是直接从缓存中获取数据。且配合 Spring Cache 使用，大幅度简化了 redis 的操作。

4.商品管理

进入商品页面，会进行分页查询，返回包含产品名称、商品价格、商品库存等信息，且支持新增、删除、修改商品，其分页展示示意图和新增示意图如图 6.29 和图 6.30 所示：

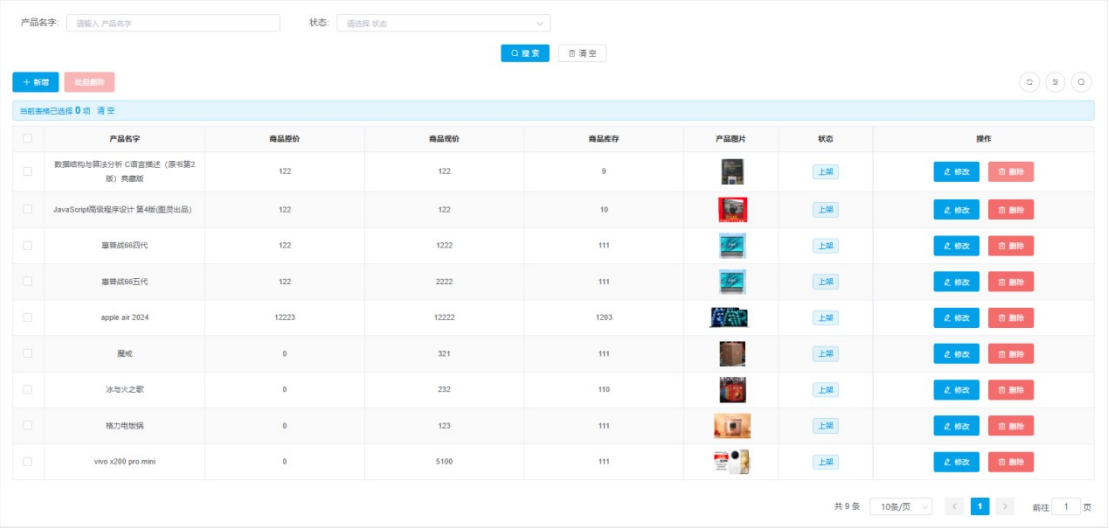

图 6.29 分页展示页面图

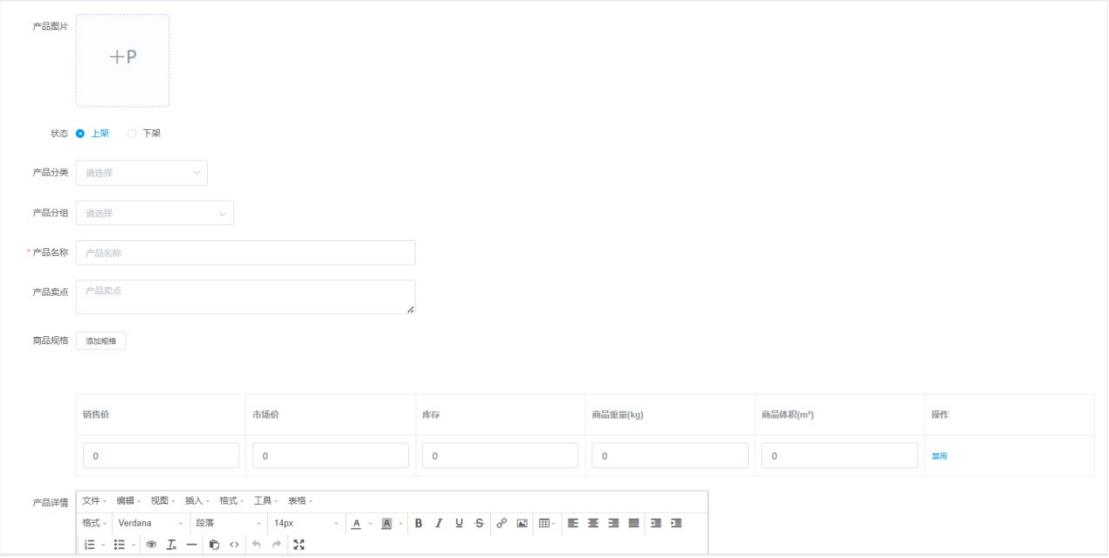

图 6.30 新增页面图

核心代码如下：

```
public Result<Page<Prod>> loadProdPage(@RequestParam Long current,
@RequestParam Long size,
@RequestParam String prodName,
@RequestParam  Long status) {
// 创建商品分页对象
```

```
Page<Prod> page = new Page<>(current,size);
// 多条件分页查询商品
page = prodService.page(page,new
LambdaQueryWrapper<Prod>().eq(ObjectUtil.isNotNull(status),Prod::getStatus,st
atus).like(StringUtils.hasText(prodName),Prod::getProdName,prodName).orderBy
Desc(Prod::getCreateTime));
return Result.success(page);
}
```

6.4.6 门店管理模块设计

进入门店管理模块，主要展示以下四个功能单元，即轮播图管理、地址管理、热搜管理和公告管理。具体页面和实现如下所示。

1.轮播图管理

在轮播图管理页面，用户可以通过状态查询图片，支持新增轮播图、删除轮播图、修改轮播图，且轮播图可以关联商品。

具体页面如图 6.31 和图 6.32 所示：

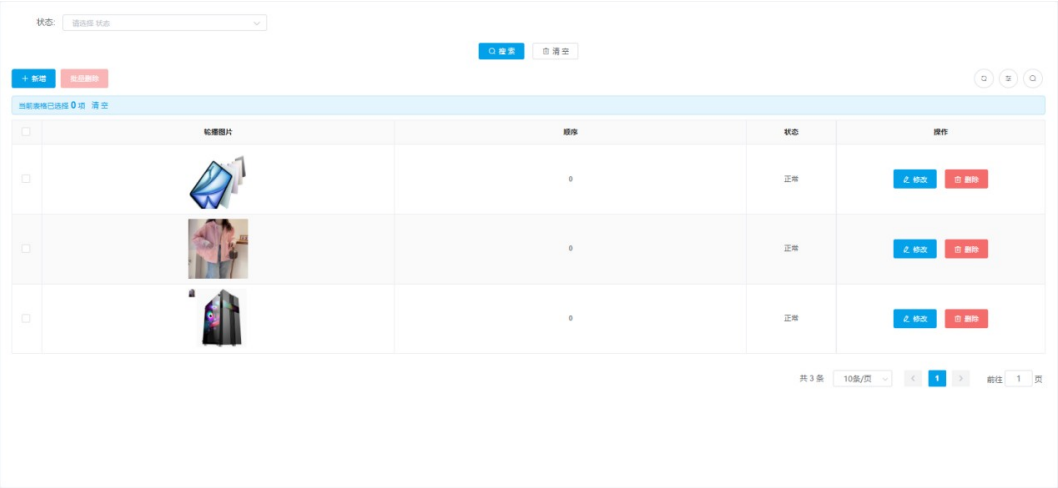

图 6.31 轮播图管理页面图

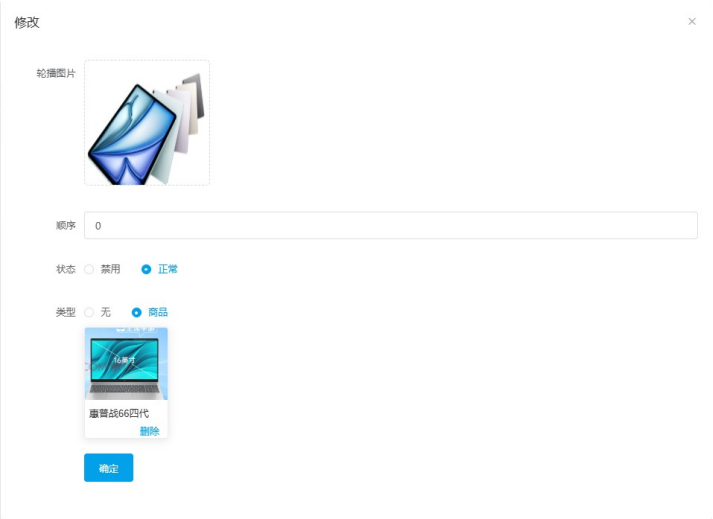

图 6.32 修改页面图

其核心代码如下：

```
@GetMapping("page")
public Result<Page<IndexImg>> loadIndexImgPage(@RequestParam Long current,
@RequestParam Long size,
@RequestParam(required = false) Integer status) {
// 创建轮播图分页对象
Page<IndexImg> page = new Page<>(current,size);
// 多条件分页查询轮播图
page = indexImgService.page(page,new LambdaQueryWrapper<IndexImg>().eq(ObjectUtil.isNotNull(status),IndexImg::getStatus,status).orderByDesc(IndexImg::getSeq));
return Result.success(page);
}
```

2.公告管理

在公告管理页面，用户可以通过标题、状态查询公告，支持新增公告、删除公告、修改公告信息。

如图 6.33 所示：

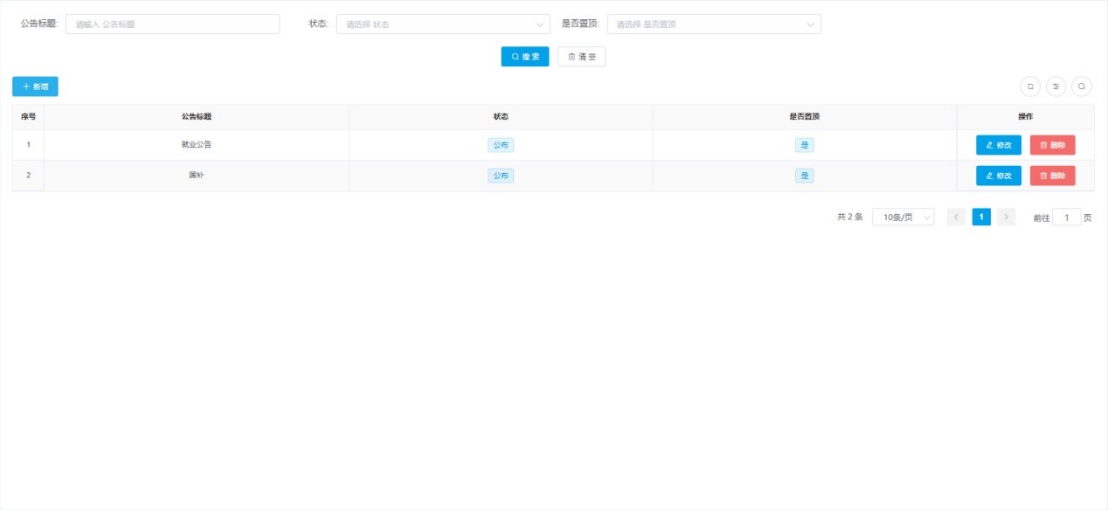

图 6.33 公告管理页面图

其核心代码如下：

```
public Result<Page<Notice>> loadNoticePage(@RequestParam Long current,
@RequestParam Long size,
@RequestParam(required = false) String title,
@RequestParam(required = false) Integer status,
@RequestParam(required = false) Integer isTop) {
// 创建公告分页对象
Page<Notice> page = new Page<>(current,size);
// 多条件分页查询公告列表
page = noticeService.page(page,new LambdaQueryWrapper<Notice>().
eq(ObjectUtil.isNotNull(status),Notice::getStatus,status)
.eq(ObjectUtil.isNotNull(isTop),Notice::getIsTop,isTop)
.like(StringUtils.hasText(title) ,Notice::getTitle,title)
.orderByDesc(Notice::getCreateTime)
);
return Result.success(page);
```

3.热搜管理

在热搜管理页面，用户可以通过标题、内容、状态查询热搜，支持新增热搜、删除热搜、修改热搜。

如图 6.34 所示：

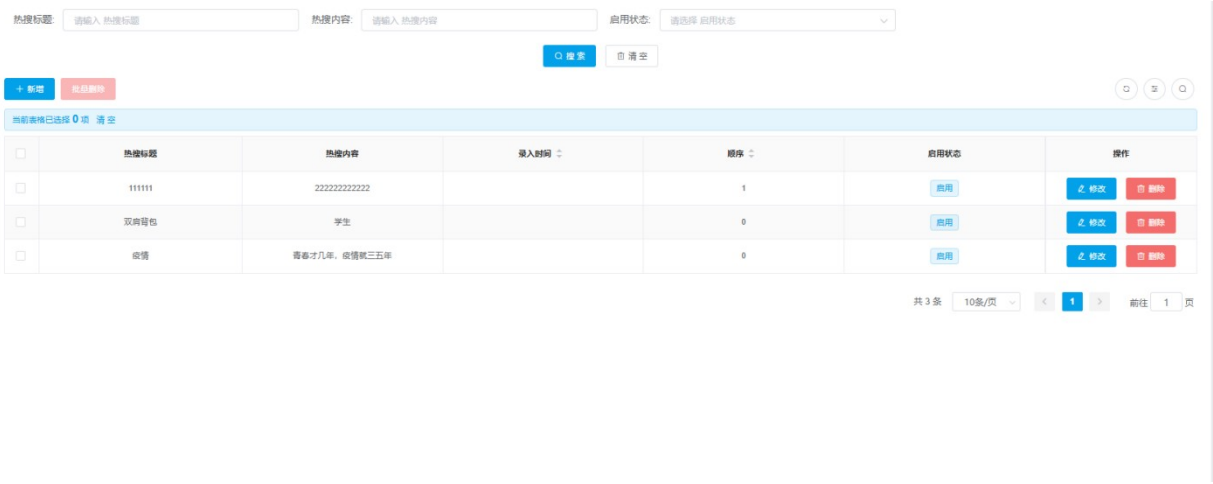

图 6.34 热搜管理

其核心代码如下：

```
@GetMapping("page")
public Result<Page<HotSearch>> getHotSearchPage(@RequestParam Long
current,
@RequestParam Long size,
@RequestParam(required = false) String title,
@RequestParam(required = false) String content,
@RequestParam(required = false) Integer status) {
Page<HotSearch> page = new Page<>(current, size);
Page<HotSearch> page1 = hotService.page(page, new
LambdaQueryWrapper<HotSearch>().eq(ObjectUtil.isNotNull(status),
HotSearch::getStatus, status)
.eq(ObjectUtil.isNotNull(status), HotSearch::getTitle, title)
.eq(ObjectUtil.isNotNull(status), HotSearch::getContent, content)
  .orderByDesc(HotSearch::getCreateTime));
return Result.success(page1);
}
```

4.地址管理

在地址管理页面，地址信息以树形结构展示，最多支持三层层级（如省-市-区）。用户可以对任意节点进行修改、添加或删除操作。修改时，用户可以更新

节点名称或详细信息；添加时，可在任意层级下新增子节点；删除时，系统会提示确认并同步移除其子节点。同时，支持展开/折叠节点，其示意图所 6.35 所示：

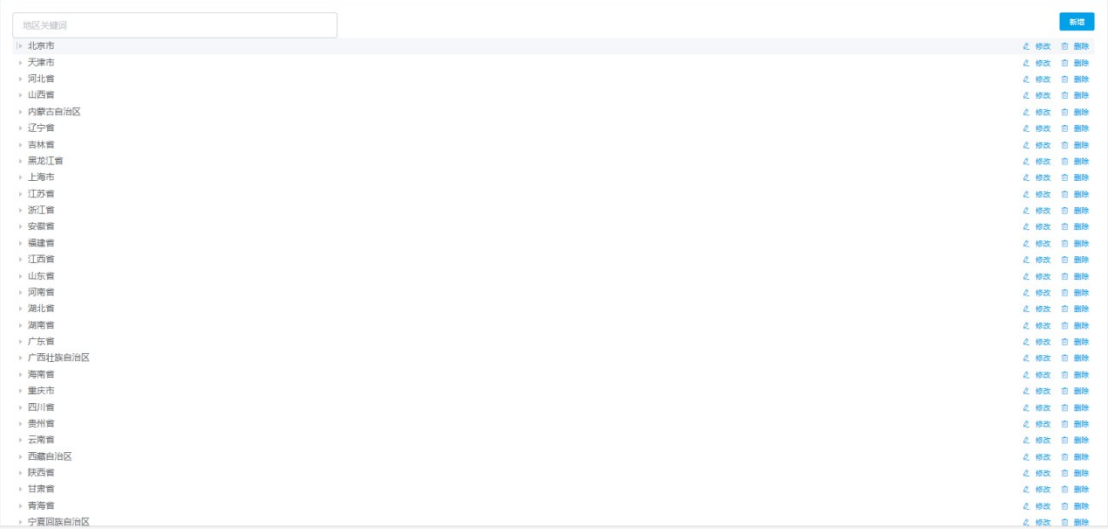

图 6.35 地址管理页面图

其核心代码如下所示：

```
@ApiOperation("查询全国地区列表")
@GetMapping("list")
@PreAuthorize("hasAuthority('admin:area:list')")
public Result<List<Area>> loadAllAreaList() {
List<Area> list = areaService.queryAllAreaList();
return Result.success(list);
}
@ApiOperation("根据地区父节点标识查询子节点集合")
@GetMapping("/listByPid")
public Result<List<Area>> a(@RequestParam String t,@RequestParam Long pid)
{
List<Area> list = areaService.list(new
LambdaQueryWrapper<Area>().eq(Area::getParentId, pid));
return Result.success(list);
}
```

在 Java 中实现树形结构的地址管理，可以通过数据库中的递归模型来存储层级关系。设计一个地址表 Area，包含字段如 id（唯一标识）、parent_id（指向父节点）、name（节点名称）。parent_id 为 NULL 表示顶级节点，非 NULL 则指向父节点的 id。通过这种设计，可以支持任意层级的结构，且方便查询、更新和删除操作。删除节点时，通过递归删除子节点。

7.总结与展望

在本次基于 Java 微服务架构的购物商城系统设计与开发过程中，笔者通过对电商平台现状与发展需求的分析，结合微服务架构和微信小程序开发，成功设计并实现了一个购物商城系统。系统采用了前后端分离的开发模式，前端利用微信小程序实现用户交互，后端则采用 SpringBoot 与 SpringCloud 技术栈，结合微服务架构，极大地提高了系统的可扩展性与稳定性。

在开发过程中，笔者深入学习并掌握了 SpringCloud、RabbitMQ、MyBatis-Plus 等技术。通过使用 SpringCloud 组件，系统得以通过网关分发请求，减轻了后端服务器的负担；利用 RabbitMQ 作为消息队列，确保了消息的高效传递，提升了系统的响应速度与稳定性。

系统的需求分析和数据库设计阶段为后续开发奠定了坚实的基础。通过对商城功能与非功能需求的详细分析，确保了系统能够处理复杂的业务逻辑。同时，数据库设计充分考虑了系统的扩展性与数据一致性，确保了系统的稳定运行。

在具体实现阶段，笔者通过设计微服务架构及各业务模块，展示了微信小程序与后端管理平台的有效协同工作。微信小程序为用户提供了简洁直观的购物体验，而后端管理平台则使得管理员能够高效地管理商品、用户及订单等信息。

开发过程中也遇到了一些技术难题，尤其是在权限认证模块的设计与实现上。最初尝试使用 Shiro 或 Spring Security 框架，但由于其复杂的配置，导致上手困难。最终，通过查阅资料，笔者选择了 Spring Security 框架，并成功实现了简洁高效的权限认证功能，解决了权限管理的难题。

在系统开发过程中，笔者对 SpringCloud、MybatisPlus、RabbitMQ 有了更深入的了解，还了解了新的权限框架 Spring Security。学习了如何将教科书知识转化为实际操作，并提高了面向对象编程技能。但也深刻意识到自己的不足，在未来的工作中，笔者将继续学习和丰富自己的专业知识。

参考文献

[1] 张素娟. 曾强:领跑实华开商务之路[J]. 中国电子商务,2004,(11):25-29.

[2] 邓加. 电子商务背景下网络购物的安全问题分析[J]. 技术与市场，2016,23(07):405.

[3] Walls C. Spring Boot in action[M]. Simon and Schuster, 2015.

[4] 杨开振. 深入浅出 Spring Boot 2.x[M]. 人民邮电出版社, 201808.442.

[5] 冯志勇,徐砚伟,薛霄,等. 微服务技术发展的现状与展望[J]. 计算机研究与发展,2020,57(05):1103-1122.

[6] 郑天民. 微服务设计原理与架构[M]. 人民邮电出版社，2018.

[7] Christudas B. Springcloud[J]. Practical Microservices Architectural Patterns: Event-Based Java Microservices with Spring Boot and Spring Cloud, 2019: 183-244.

[8] DuBois P. MySQL[M]. Addison-Wesley, 2013.

[9] 张蒙蒙,曹成茂. 大数据时代 Redis 缓存的性能分析与优化[J]. 巢湖学院学报,2022,24(03):80-87+96.

[10] 王晓燕. Redis 基本数据类型的使用[J]. 办公自动化, 2020, 25(23): 48-49+58.

[11] 张俊. 基于 Redis 实现关系型数据库内存化研究[D]. 四川师范大学,2021.DOI:10.27347/d.cnki.gssdu.2021.000619

[12] 唐权,周蓉,张勇. RabbitMQ 消息中间件在 Spring Boot 教学中的应用[J]. 现代信息科学,2020,4(18):125-127.DOI:10.19850/j.cnki.2096-4706.2020.18.035.

[13] 李宇杰. 基于微服务架构的电商微信小程序的设计与实现[D]. 华东师范大学,2023. DOI:10.27149/d.cnki.ghdsu.2023.003555

[14] 王松. 深入浅出 Spring Security. 清华大学出版社,2021.

[15] 吕玉桂. Spring Security+JWT 实现微服务架构中的身份验证和授权[J]. 电脑知识与技术,2024,20(22):60-63.DOI:10.14004/j.cnki.ckt.2024.1219.

[16] 殷佳庭,陆婷婷. MinIO 对象存储技术的集成开发方法研究[J]. 芜湖职业技术学院学报,2024,26(03):36-39.

[17] 张海藩，牟永敏. 软件工程导论[M]. 北京：清华大学出版社，2013.8.

[18] 陈瑞. 基于 Springboot 高并发 Java Web 开发模式[J]. 电脑编程技巧与维护,2019,(04):27-30.DOI:10.16184/j.cnki.comprg.2019.04.009.

[19] 刘欢. 基于微服务架构的商城购物系统的设计与实现[D]. 西安电子科技大学,2021.DOI:10.27389/d.cnki.gxadu.2021.003338.

[20] 莫竣成,田秀云. 基于 Java 的网上购物平台系统设计[J]. 机电工程技

术,2021,50(01):103-105.